

# MLIR: A Compiler Infrastructure for the End of Moore’s Law

Chris Lattner \*  
Google

Mehdi Amini  
Google

Uday Bondhugula  
IISc

Albert Cohen  
Google

Andy Davis  
Google

Jacques Pienaar  
Google

River Riddle  
Google

Tatiana Shpeisman  
Google

Nicolas Vasilache  
Google

Oleksandr Zinenko  
Google

## Abstract

This work presents MLIR, a novel approach to building reusable and extensible compiler infrastructure. MLIR aims to address software fragmentation, improve compilation for heterogeneous hardware, significantly reduce the cost of building domain specific compilers, and aid in connecting existing compilers together. MLIR facilitates the design and implementation of code generators, translators and optimizers at different levels of abstraction and also across application domains, hardware targets and execution environments. The contribution of this work includes (1) discussion of MLIR as a research artifact, built for extension and evolution, and identifying the challenges and opportunities posed by this novel design point in design, semantics, optimization specification, system, and engineering. (2) evaluation of MLIR as a generalized infrastructure that reduces the cost of building compilers—describing diverse use-cases to show research and educational opportunities for future programming languages, compilers, execution environments, and computer architecture. The paper also presents the rationale for MLIR, its original design principles, structures and semantics.

## 1 Introduction

Compiler design is a mature field with a wide range of well-known algorithms, with applications to code generation, static analysis, program transformation, and more. The field also has seen the development of a number of mature technology platforms which have enabled massive reuse across the compiler community, including systems like the LLVM compiler infrastructure [25], the Java Virtual Machine (JVM) [26], and many others. A common characteristic of these popular systems is their “one size fits all” approach—a *single abstraction level* to interface with the system: the LLVM Intermediate Representation (IR) is roughly “C with vectors”, and JVM provides an “object-oriented type system with a garbage collector” abstraction. This “one size fits all” approach is incredibly valuable—and in practice, the mapping to these domains from ubiquitous source languages (C/C++ and Java respectively) is straightforward.

At the same time, many problems are better modeled at a higher- or lower-level abstraction, e.g. source-level analysis of C++ code is very difficult on LLVM IR. We observe that many languages (including e.g. Swift, Rust, Julia, Fortran) develop their own IR in order to solve domain-specific problems, like language/library-specific optimizations, flow-sensitive type checking (e.g. for linear

\*With SiFive at the time of publication.

# MLIR：摩尔定律终结时代的编译器基础设施

克里斯·拉特纳 \* 谷歌 迈赫迪·阿米尼 谷歌 乌代·邦杜古拉 印度科学理工学院 阿尔贝·科昂 谷歌 安迪·戴维斯 谷歌 雅克·皮纳尔 谷歌 里弗·里德尔 谷歌 塔季扬娜·什佩斯曼 谷歌 尼古拉·瓦西拉凯 谷歌 奥列克桑德尔·齐年科 谷歌

## 摘要

本工作提出了MLIR，一种构建可复用、可扩展编译器基础设施的新方法。MLIR致力于解决软件碎片化问题，提升异构硬件编译效率，大幅降低构建领域专用编译器的成本，并助力现有编译器间的互联互通。MLIR 促进了在不同抽象层次以及跨应用领域、硬件目标和执行环境中设计和实现代码生成器、翻译器和优化器。本工作的贡献包括：(1) 将 MLIR 作为一种研究工具进行讨论，该工具为扩展和演化而构建，并识别了这种新颖设计点在设计、语义、优化规范、系统和工程方面带来的挑战与机遇。(2) 将 MLIR 作为一种通用基础设施进行评估，以降低构建编译器的成本——通过描述多样化的用例，展示未来编程语言、编译器、执行环境和计算机架构的研究与教育机遇。本文还阐述了 MLIR 的基本原理、其原始设计原则、结构和语义。

## 1 引言

编译器设计是一个成熟的领域，拥有众多广为人知的算法，应用于代码生成、静态分析、程序转换等方面。该领域还发展出了多个成熟的技术平台，促进了编译器社区内的大规模复用，例如LLVM编译器基础设施[25]、Java虚拟机（JVM）[26]等系统。这些流行系统的一个共同特点是采用“一刀切”的方法——通过一个*single abstraction level*与系统交互：LLVM中间表示（IR）大致是“带向量的C语言”，而JVM则提供“带有垃圾回收器的面向对象类型系统”的抽象。这种“一刀切”的方法极具价值——在实践中，从普遍使用的源语言（分别为C/C++和Java）到这些领域的映射是直接明了的。

与此同时，许多问题在更高或更低层次的抽象上建模更为合适，例如，在LLVM IR上对C++代码进行源码级分析非常困难。我们注意到，许多语言（包括Swift、Rust、Julia、Fortran等）都开发了自己的中间表示（IR），以解决特定领域的问题，如语言/库特定的优化、流敏感的类型检查（例如用于线性

\*With SiFive at the time of publication.

0  
2  
0  
2  
r  
a  
M  
1  
J  
L  
P  
·  
s  
c  
[  
2  
v  
4  
5  
0  
1  
1  
2  
0  
0  
2  
:  
v  
i  
X  
r  
a

types), and to improve the implementation of the lowering process. Similarly, machine learning systems typically use “ML graphs” as a domain-specific abstraction in the same way.

While the development of domain specific IRs is a well studied art, their engineering and implementation cost remains high. The quality of the infrastructure is not always a first priority (or easy to justify) for implementers of these systems. Consequently, this can lead to lower quality compiler systems, including user-visible problems like slow compile times, buggy implementations, suboptimal diagnostic quality, poor debugging experience for optimized code, etc.

The MLIR project aims to directly tackle these programming language design and implementation challenges—by making it very cheap to define and introduce new abstraction levels, and provide “in the box” infrastructure to solve common compiler engineering problems. MLIR does this by (1) standardizing the Static Single Assignment (SSA)-based IR data structures, (2) providing a declarative system for defining *IR dialects*, and (3) providing a wide range of common infrastructure (including documentation, parsing and printing logic, location tracking, multithreaded compilation support, pass management, etc).

This paper explores various design points of the MLIR system, relates our experience applying it to a number of different problems, and discusses implications this work may have for language design and education.

### 1.1 Contributions

While most of the MLIR system is built out of well known compiler algorithms, the design points are sufficiently novel that it provides opportunities for interesting research. The contributions of this paper are:

- a description of a novel compiler infrastructure with important industrial and research applications;
- new approaches to building scalable and modular compiler systems;
- exploration of selected applications of MLIR to diverse domains, illustrating the generality of the system;
- shared experience developing systems that build on the MLIR infrastructure.

### 1.2 Where did MLIR come from?

Work on MLIR began with a realization that modern machine learning frameworks are composed of many different compilers, graph technologies, and runtime systems (see Figure 1)—which did not share a common infrastructure or design point, and not all of which were following best practices in compiler design. This manifested in multiple user-visible ways, including poor error messages, failures in edge cases, unpredictable performance, and difficulty generalizing the stack to support new hardware.

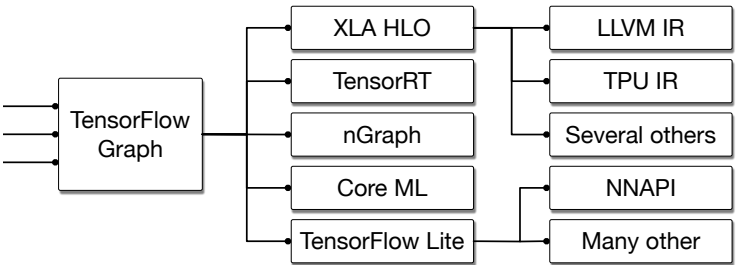


Figure 1: TensorFlow model execution spanning different frameworks.

We soon realized that the compiler industry as a whole has a similar problem: existing systems like LLVM are very successful at unifying and integrating work across a range of different language implementations, but modern high level languages often end up building their own high-level IR and reinventing a lot of the same kinds of technology for higher levels of abstraction (see Figure 2). At the same time, the LLVM community frequently struggled with questions about how to best represent

类型），并改进降低过程的实现。同样，机器学习系统通常以相同的方式使用“ML图”作为领域特定的抽象。

虽然领域特定IR的开发是一门经过深入研究的艺术，但其工程和实现成本仍然很高。对于这些系统的实现者来说，基础设施的质量并不总是首要任务（或容易证明其合理性）。因此，这可能导致编译器系统质量较低，包括用户可见的问题，如编译时间缓慢、实现存在漏洞、诊断质量欠佳、优化代码的调试体验差等。

MLIR项目旨在直接应对这些编程语言设计与实现中的挑战——通过大幅降低定义和引入新抽象层级的成本，并提供“开箱即用”的基础设施来解决常见的编译器工程问题。MLIR通过以下方式实现这一目标：(1) 标准化基于静态单赋值（SSA）的中间表示数据结构，(2) 提供用于定义*IR dialects*的声明式系统，(3) 提供广泛的基础设施支持（包括文档系统、解析与打印逻辑、位置追踪、多线程编译支持、编译流程管理等）。

本文探讨了MLIR系统的多种设计要点，分享了我们在将其应用于多个不同问题时的经验，并讨论了这项工作对语言设计和教育可能产生的影响。

### 1.1 贡献

虽然MLIR系统的大部分构建于众所周知的编译器算法之上，但其设计理念具有足够的新颖性，为有趣的研究提供了机会。本文的贡献在于：

- 一种具有重要工业和研究应用价值的新型编译器基础设施的描述；
- 构建可扩展和模块化编译器系统的新方法；
- 探索MLIR在多个领域的精选应用，展示该系统的通用性；
- 基于MLIR基础设施开发系统的共享经验。

### 1.2 MLIR从何而来？

MLIR的工作始于一个认识：现代机器学习框架由许多不同的编译器、图技术和运行时系统组成（见图1）——它们没有共享共同的基础设施或设计点，且并非所有都遵循编译器设计的最佳实践。这以多种用户可见的方式表现出来，包括糟糕的错误信息、边缘情况下的故障、不可预测的性能，以及难以推广该堆栈以支持新硬件。

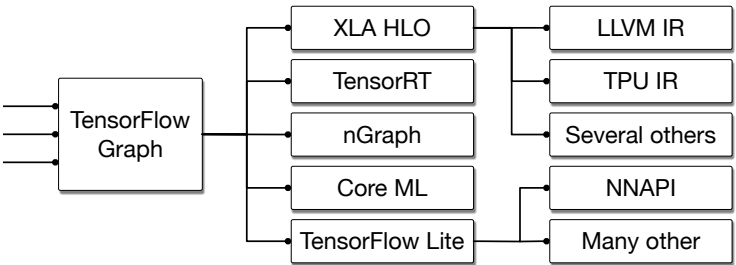


图1：跨不同框架的TensorFlow模型执行。

我们很快意识到整个编译器行业也存在类似的问题：像LLVM这样的现有系统在统一和整合多种不同语言实现方面非常成功，但现代高级语言往往最终会构建自己的高级中间表示，并在更高抽象层次上重复发明大量同类技术（见图2）。与此同时，LLVM社区经常为如何最佳表示

parallel constructs, how to share implementation of common front-end lowering infrastructure (e.g. for C calling conventions, or cross-language features like OpenMP) with no satisfactory solutions being available.

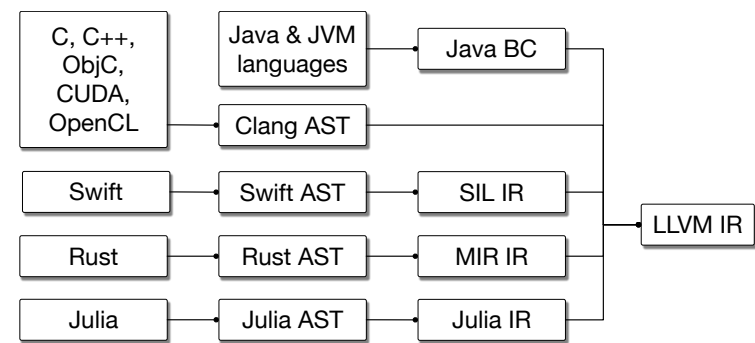


Figure 2: Compilation pipeline of different languages with multiple mid-level IRs for language-specific optimization with common backend for multiple hardware targets.

Faced with this challenge and perspective, we decided that we could not afford the engineering effort to implement  $N$  improved compiler instances, and we needed to build a more general solution. We reasoned that this would give us the ability to invest in one set of high quality infrastructure which would benefit multiple domains, would allow us to progressively upgrade existing systems in place, would make it easier for us to tackle pressing problems like heterogeneous compilation for specialized accelerators, and would provide interesting research opportunities to define and explore.

Now that we have a significant amount of experience building and deploying MLIR-based systems, we are able to look back on the rationale and design of the infrastructure, and discuss why this direction was pursued.

## 2 Design Principles

Let us now explore the requirements that guided the design of MLIR.

**Little builtin, everything customizable** The system is based on a minimal number of fundamental concepts, leaving most of the intermediate representation fully customizable. A handful of abstractions—types, operations and attributes, which are the most common in IRs—should be used to express everything else, allowing fewer and more consistent abstractions that are easy to comprehend, extend and adopt. Broadly, customizability ensures the system can adapt to changing requirements and is more likely to be applicable to future problems. In that sense, we ought to build an IR as a rich infrastructure with reusable components and programming abstractions supporting the syntax and semantics of its intermediate language.

A success criterion for customization is the possibility to express a diverse set of abstractions including machine learning graphs, ASTs, mathematical abstractions such as polyhedral, Control Flow Graphs (CFGs) and instruction-level IRs such as LLVM IR, all without hard coding concepts from these abstractions into the system.

Certainly, customizability creates a risk of internal fragmentation due to poorly compatible abstractions. While there is unlikely a purely technical solution to the ecosystem fragmentation problem, the system should encourage one to design reusable abstractions and assume they will be used outside of their initial scope.

**SSA and regions** The Static Single Assignment (SSA) form [15] is a widely used representation in compiler IRs. It provides numerous advantages including making dataflow analysis simple and sparse, is widely understood by the compiler community for its relation with continuation-passing style, and is established in major frameworks. While many existing IRs use a flat, linearized CFG, representing higher level abstractions push introducing *nested regions* as a first-class concept in the IR. This goes beyond the traditional region formation to lift higher level abstractions (e.g., loop trees), speeding up the compilation process or extracting instruction, or SIMD parallelism [22, 21, 37]. To

并行构造，如何共享通用前端降级基础设施（例如针对C调用约定，或如OpenMP这类跨语言特性）的实现，目前尚无令人满意的解决方案。

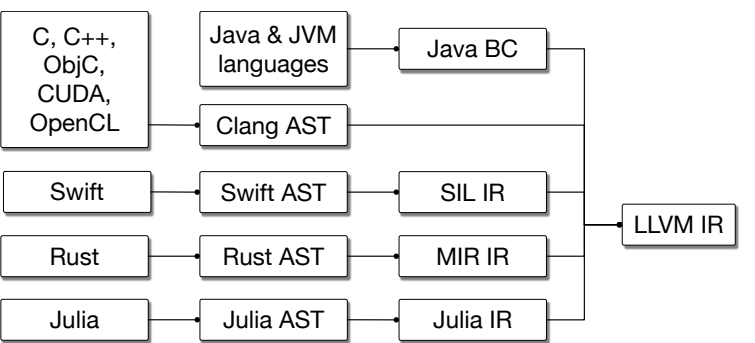


图2: 不同语言的编译流水线，采用多级中间表示进行语言特定优化，并共享后端以支持多种硬件目标。

面对这一挑战和前景，我们判断自己无法承担实现 $N$ 改进型编译器实例所需的工程投入，因此需要构建一个更通用的解决方案。我们认为，这将使我们有能力投资于一套高质量的基础设施，这套设施将惠及多个领域，使我们能够逐步升级现有系统，更轻松地对专用加速器异构编译等紧迫问题，并为定义和探索提供有趣的研究机会。

在构建和部署基于MLIR的系统方面积累了丰富经验后，我们现在能够回顾该基础设施的设计理念与决策依据，并探讨当初选择这一发展方向的原因。

## 2 设计原则

现在，让我们探讨指导MLIR设计的需求。

小巧的内建，一切皆可定制 该系统基于最少数量的基本概念构建，使得大部分中间表示层完全可定制。少数抽象——类型、操作和属性（这些在中间表示中最常见）——应被用于表达其他所有内容，从而实现更少且更一致的抽象，易于理解、扩展和采用。从广义上讲，可定制性确保了系统能够适应不断变化的需求，并更有可能适用于未来的问题。从这个意义上说，我们应当将中间表示构建为一个丰富的基础设施，包含可复用组件和编程抽象，以支持其中间语言的语法和语义。

定制化的一个成功标准是能够表达多样化的抽象集合，包括机器学习图、抽象语法树（ASTs）、多面体等数学抽象、控制流图（CFGs）以及LLVM IR等指令级中间表示，所有这些都无需将这些抽象的概念硬编码到系统中。

当然，可定制性带来了因抽象兼容性差而导致内部碎片化的风险。虽然不太可能有纯粹的技术方案来解决生态系统碎片化问题，但系统应鼓励设计可复用的抽象，并假设它们将在初始范围之外得到使用。

SSA与区域 静态单赋值SSA形式[15]是编译器中间表示中广泛使用的表达方式。它具有诸多优势，包括简化数据流分析、实现稀疏化处理，且因其与延续传递风格的关联性而深受编译器社区认可，并已在主流框架中确立地位。现有许多中间表示采用扁平化、线性化的控制流图，而表达更高层次的抽象则需引入*nested regions*作为中间表示中的一等概念。这超越了传统的区域构建方式，旨在提升更高层次的抽象（例如循环树结构），从而加速编译流程、优化指令提取或实现SIMD并行化处理[22, 21, 37]。

support heterogeneous compilation, the system has to support the expression of structured control flow, concurrency constructs, closures in source languages, and many other purposes. One specific challenge is to make CFG-based analyses and transformations compose over nested regions.

In doing so, we aim to sacrifice the normalization, and sometimes the canonicalization properties of LLVM. Being able to lower a variety of data and control structures into a smaller collection of normalized representations is key to keeping compiler complexity under control. The canonical loop structure with its pre-header, header, latch, body, is a prototypical case of a linearized control flow representation of a variety of loop constructs in front-end languages. We aim at offering users a choice: depending on the compilation algorithm of interest, of the pass in the compilation flow, nested loops may be captured as nested regions, or as linearized control flow. By offering such a choice, we depart from the normalization-only orientation of LLVM while retaining the ability to deal with higher level abstractions when it matters. In turn, leveraging such choices raises questions about how to control the normalization of abstractions, which is the purpose of the next paragraph.

**Progressive lowering** The system should support *progressive lowering*, i.e. from the higher-level representation down to the lowest-level, with the lowering being performed in small steps along multiple abstraction levels. The need for multiple levels of abstractions stems from the variety of platforms and programming models that a generic compiler infrastructure has to support.

Previous compilers have been introducing multiple fixed levels of abstraction in their pipeline—e.g. the Open64 WHIRL representation [30] has five levels, as does the Clang compiler which lowers from ASTs to LLVM IR, to SelectionDAG, to MachineInstr, and to MCInst. Whereas these approaches are done in a rigid way, more flexible designs are required to support extensibility.

This has deep implications on the phase ordering of transformations. As compiler experts started implementing more and more transformation passes, complex interactions between these passes started appearing. It was shown early on that combining optimization passes allows the compiler to discover more facts about the program. One of the first illustrations of the benefits of combining passes was to mix constant propagation, value numbering and unreachable code elimination [13]. More generally, compiler passes can be roughly categorized into four roles: (1) optimizing transformations, (2) enabling transformations, (3) lowering and (4) cleanup. The system should allow for mixing and matching these roles at the granularity of a single operation rather than sequencing passes on the full compilation unit.

**Maintain higher-level semantics** The system needs to retain higher-level semantics and structure of computations that are required for analysis or optimizing performance. Attempts to raise semantics once lowered are fragile and shoe-horning this information into a low-level often invasive (e.g., all passes need to be verified/revisited in the case of using debug information to record structure). Instead, the system should maintain structure of computation and progressively lower to the hardware abstraction. The loss of structure is then conscious and happens only where the structure is no longer needed to match the underlying execution model. For example, the system should preserve the structured control flow such as loop structure throughout the relevant transformations; removing this structure, i.e. going to CFG-based control flow, essentially means no further transformations will be performed on this level. The state of the art in modeling parallel computing constructs in a production compiler highlights how difficult the task may be in general [23, 42].

As a corollary, mixing different levels of abstractions and different concepts in the same IR is a key property of the system to allow a part of the representation to remain in higher-level abstraction while another part is lowered. This would enable, for instance, a compiler for a custom accelerator to reuse some higher-level structure and abstractions defined by the system alongside with primitive scalar/vector instructions specific to the accelerator.

**IR validation** The openness of the ecosystem calls for an extensive validation mechanism. While verification and testing are useful to detect compiler bugs, and to capture IR invariants, the need for robust validation methodologies and tools is amplified in an extensible system. The mechanism should aim to make this easy to define and as declarative as practical, providing a single source of truth.

支持异构编译，系统必须能够表达结构化控制流、并发构造、源语言中的闭包以及许多其他用途。一个具体的挑战是使基于CFG的分析和转换能够在嵌套区域上组合。

为此，我们旨在牺牲LLVM的规范化特性，有时甚至是其标准化特性。能够将各种数据和控制结构降级为更小集合的规范化表示，是控制编译器复杂性的关键。具有前导块、头部、锁存块和主体的规范循环结构，是前端语言中各种循环构造线性化控制流表示的典型例子。我们的目标是为用户提供选择：根据感兴趣的编译算法、编译流程中的阶段，嵌套循环可以被捕获为嵌套区域，或作为线性化控制流。通过提供这种选择，我们偏离了LLVM仅追求规范化的方向，同时保留了在必要时处理更高级抽象的能力。反过来，利用这些选择引发了如何控制抽象规范化的问题，这正是下一段的目的。

渐进式降级 系统应支持*progressive lowering*，即从高级表示逐步降至最低级，降级过程需在多个抽象层次上以小步骤进行。多级抽象的必要性源于通用编译器基础设施需要支持多种平台和编程模型。

以往的编译器在其流水线中引入了多个固定的抽象层次——例如，Open64 WHIRL表示法[30]有五个层次，Clang编译器也是如此，它从AST降至LLVM IR，再到SelectionDAG，再到MachineInstr，最后到MCInst。尽管这些方法以僵化的方式实现，但为了支持可扩展性，需要更灵活的设计。

这对变换的相位排序有着深远的影响。随着编译器专家开始实现越来越多的变换过程，这些过程之间复杂的相互作用开始显现。早期研究表明，结合优化过程能让编译器发现更多关于程序的事实。结合过程益处的最早例证之一是将常量传播、值编号和不可达代码消除相结合[13]。更一般地说，编译器过程大致可分为四种角色：(1) 优化变换，(2) 启用变换，(3) 降级和(4) 清理。系统应允许在单个操作的粒度上混合和匹配这些角色，而不是在整个编译单元上顺序执行过程。

保持高层语义 系统需要保留用于分析或优化性能所需的高层语义和计算结构。一旦语义被降低，再试图提升往往是脆弱且牵强的，将这些信息强行塞入低层级通常具有侵入性（例如，若使用调试信息记录结构，则所有处理步骤都需要重新验证/审视）。相反，系统应保持计算结构，并逐步降低至硬件抽象层。这样，结构的丢失是有意识的，并且仅发生在结构不再需要匹配底层执行模型的地方。例如，系统应在相关转换过程中保持结构化控制流（如循环结构）；移除这种结构（即转向基于CFG的控制流）本质上意味着不再在该层级执行进一步的转换。生产编译器中建模并行计算结构的最新技术突显了这项任务在一般情况下可能有多么困难[23, 42]。

作为推论，在同一中间表示中混合不同层次的抽象和不同概念是该系统的一个关键特性，它允许部分表示保持较高层次的抽象，而另一部分则被降低。例如，这将使自定义加速器的编译器能够重用系统定义的一些高层次结构和抽象，同时结合特定于加速器的原始标量/向量指令。

IR验证 生态系统的开放性要求一个广泛的验证机制。虽然验证和测试有助于检测编译器错误并捕获IR不变量，但在可扩展系统中，对稳健验证方法和工具的需求被放大。该机制旨在使定义变得容易，并尽可能声明式，提供单一事实来源。

```

%results:2 = "d.operation"(%arg0, %arg1) ({
  // Regions belong to Ops and can have multiple blocks.
  ^block(%argument: !d.type):
    // Ops have function types (expressing mapping).
    %value = "nested.operation"() ({
      // Ops can contain nested regions.
      "d.op"() : () -> ()
    }) : () -> (!d.other_type)
    "consume.value"(%value) : (!d.other_type) -> ()
  ^other_block:
    "d.terminator"() [^block(%argument : !d.type)] : () -> ()
})
// Ops can have a list of attributes.
{attribute="value" : !d.type} : () -> (!d.type, !d.other_type)

```

Figure 3: Operation (Op) is a main entity in MLIR; operations contain a list of regions, regions contain a list of blocks, blocks contains a list of Ops, enabling recursive structures

A long term goal would be to reproduce the successes of translation validation [35, 29, 50, 51] and modern approaches to compiler testing [12]. Both are currently open problems in the context of an extensible compiler ecosystem.

**Declarative rewrite patterns** Defining representation modifiers should be as simple as that of new abstractions; a compiler infrastructure is only as good as the transformations it supports. Common transformations should be implementable as rewrite rules expressed declaratively, in a machine-analyzable format to reason about properties of the rewrites such as complexity and completion. Rewriting systems have been studied extensively for their soundness and efficiency, and applied to numerous compilation problems, from type systems to instruction selection. Since we aim for unprecedented extensibility and incremental lowering capabilities, this opens numerous avenues for modeling program transformations as rewrite systems. It also raises interesting questions about how to represent the rewrite rules and strategies, and how to build machine descriptions capable of steering rewriting strategies through multiple levels of abstraction. The system needs to address these questions while preserving extensibility and enforcing a sound, monotonic and reproducible behavior.

**Source location tracking and traceability** The provenance of an operation—including its original location and applied transformations—should be easily traceable within the system. This intends to address the lack-of-transparency problem, common to complex compilation systems, where it is virtually impossible to understand how the final representation was constructed from the original one.

This is particularly problematic when compiling safety-critical and sensitive applications, where tracing lowering and optimization steps is an essential component of software certification procedures [43]. When operating on secure code such as cryptographic protocols or algorithms operating on privacy-sensitive data, the compiler often faces seemingly redundant or cumbersome computations that embed a security or privacy property not fully captured by the functional semantics of the source program: this code may prevent the exposure of side channels or harden the code against cyber or fault attacks. Optimizations may alter or completely invalidate such protections [56]; this lack of transparency is known as WYSINWYX [6] in secure compilation. One indirect goal of accurately propagating high-level information to the lower levels is to help support secure and traceable compilation.

### 3 IR Design Details

This section describes the design of the IR in MLIR following the principles from the previous section.

```

%results:2 = "d.operation"(%arg0, %arg1) ({
  // Regions belong to Ops and can have multiple blocks.
  ^block(%argument: !d.type):
    // Ops have function types (expressing mapping).
    %value = "nested.operation"() ({
      // Ops can contain nested regions.
      "d.op"() : () -> ()
    }) : () -> (!d.other_type)
    "consume.value"(%value) : (!d.other_type) -> ()
  ^other_block:
    "d.terminator"() [^block(%argument : !d.type)] : () -> ()
})
// Ops can have a list of attributes.
{attribute="value" : !d.type} : () -> (!d.type, !d.other_type)

```

图3: 操作 (Op) 是MLIR中的一个主要实体; 操作包含一个区域列表, 区域包含一个块列表, 块包含一个操作列表, 从而支持递归结构

一个长期目标将是重现翻译验证[35, 29, 50, 51]的成功以及现代编译器测试方法[12]的成就。这两者在可扩展编译器生态系统的背景下目前仍是未解决的问题。

**声明式重写模式** 定义表示形式修改器应当与定义新抽象一样简单; 编译器基础设施的好坏取决于其支持的转换能力。常见的转换应能实现为以声明方式表达的、机器可分析的格式的重写规则, 以便推理重写的属性, 如复杂性和完成性。重写系统因其可靠性和效率已被广泛研究, 并应用于从类型系统到指令选择的众多编译问题。由于我们追求前所未有的可扩展性和增量式降级能力, 这为将程序转换建模为重写系统开辟了众多途径。它也引发了关于如何表示重写规则和策略, 以及如何构建能够引导重写策略跨越多个抽象层次的机器描述的有趣问题。系统需要在保持可扩展性并确保可靠、单调且可重现行为的同时, 解决这些问题。

**源位置追踪与可追溯性** 操作的来源——包括其原始位置 and 应用的转换——应在系统内易于追溯。这旨在解决复杂编译系统中常见的透明度不足问题, 即几乎无法理解最终表示是如何从原始表示构建而成的。

在编译安全关键型和敏感应用程序时, 这一问题尤为突出, 因为追踪降级和优化步骤是软件认证流程的重要组成部分[43]。当处理安全代码 (如加密协议或处理隐私敏感数据的算法) 时, 编译器常面临看似冗余或繁琐的计算, 这些计算嵌入了源程序功能语义未能完全捕获的安全或隐私属性: 此类代码可防止侧信道泄露, 或增强代码抵御网络攻击及故障攻击的能力。优化操作可能会改变甚至完全破坏此类保护机制[56]; 这种透明度的缺失在安全编译领域被称为WYSINWYX现象[6]。将高层级信息准确传递至底层的一个间接目标, 正是为了支持安全且可追溯的编译过程。

### 3 IR设计细节

本节根据上一节的原则, 描述了MLIR中IR的设计。



```

// Attribute aliases can be forward-declared.
#map1 = (d0, d1) -> (d0 + d1)
#map3 = () [s0] -> (s0)

// Ops may have regions attached.
"affine.for"(%arg0) ({
// Regions consist of a CFG of blocks with arguments.
^bb0(%arg4: index):
// Block are lists of operations.
"affine.for"(%arg0) ({
^bb0(%arg5: index):
// Ops use and define typed values, which obey SSA.
%0 = "affine.load"(%arg1, %arg4) {map = (d0) -> (d0)}
: (memref<?xf32>, index) -> f32
%1 = "affine.load"(%arg2, %arg5) {map = (d0) -> (d0)}
: (memref<?xf32>, index) -> f32
%2 = "std.mul"(%0, %1) : (f32, f32) -> f32
%3 = "affine.load"(%arg3, %arg4, %arg5) {map = #map1}
: (memref<?xf32>, index, index) -> f32
%4 = "std.add"(%3, %2) : (f32, f32) -> f32
"affine.store"(%4, %arg3, %arg4, %arg5) {map = #map1}
: (f32, memref<?xf32>, index, index) -> ()
// Blocks end with a terminator Op.
"affine.terminator"() : () -> ()
// Ops have a list of attributes.
}) {lower_bound = () -> (0), step = 1 : index, upper_bound = #map3}
: (index) -> ()
"affine.terminator"() : () -> ()
}) {lower_bound = () -> (0), step = 1 : index, upper_bound = #map3}
: (index) -> ()

```

Figure 4: MLIR *generic* representation for polynomial multiplication using affine and std dialects. The same IR is displayed with the custom syntax Figure 8.

**Operations** The unit of semantics in MLIR is an “operation”, referred to as *Op*. Everything from “instruction” to “function” to “module” are modeled as Ops in this system. MLIR does not have a fixed set of Ops, but allows (and encourages) user-defined extensions—compiler passes treat unknown Ops conservatively, and MLIR has rich support for describing the semantics of Ops to passes through traits, privileged operation hooks and optimization interfaces as described in Section 6.1.

Ops (see Figure 3) have a unique opcode, which, textually, is a string identifying its dialect and the operation. Ops take and produce zero or more *values*, called *operands* and *results* respectively, and these are maintained in SSA form. All values have a type, similarly to LLVM IR. In addition to an opcode, operands and results, Ops may also have *Attributes*, *Regions*, *Block Arguments*, and *Location Information* as well. Figure 4 illustrates values and Ops, %-identifiers are (packs of) named values, with “:” specifying the number in a pack if more than one and “#” a particular value. In the generic textual representation, operation names are quoted string literals followed by operands in parentheses.

**Attributes** An MLIR attribute is structured compile-time static information, e.g., integer constant values, string data, or a list of constant floating point values. Attributes are typed, and each Op instance has an open key-value dictionary from string names to attribute values. In the generic syntax, attributes are between the Op operands and its type as a brace-enclosed comma-separated list of key-value pairs. For example, Figure 4 uses attributes to define bounds of a loop that are known to be constant affine forms: {lower\_bound = () -> (0), step = 1 : index, upper\_bound = #map3} where lower\_bound, upper\_bound and step are attribute names. The () -> (0) notation is used for inline affine forms, in this case producing an affine function producing a constant 0 value. The #map3 notation is used for attribute *aliases*, which allow one to associate an attribute value with a label upfront and use the label anywhere an attribute value is expected.

As with opcodes, there is no fixed set of attributes. Attributes derive their meaning either from the Op semantics or from the dialect (Section 3) they are associated with. Attributes are also extensible, allowing direct references to foreign data structures, which is useful for integrating with existing

```

// Attribute aliases can be forward-declared.
#map1 = (d0, d1) -> (d0 + d1)
#map3 = () [s0] -> (s0)

// Ops may have regions attached.
"affine.for"(%arg0) ({
// Regions consist of a CFG of blocks with arguments.
^bb0(%arg4: index):
// Block are lists of operations.
"affine.for"(%arg0) ({
^bb0(%arg5: index):
// Ops use and define typed values, which obey SSA.
%0 = "affine.load"(%arg1, %arg4) {map = (d0) -> (d0)}
: (memref<?xf32>, index) -> f32
%1 = "affine.load"(%arg2, %arg5) {map = (d0) -> (d0)}
: (memref<?xf32>, index) -> f32
%2 = "std.mul"(%0, %1) : (f32, f32) -> f32
%3 = "affine.load"(%arg3, %arg4, %arg5) {map = #map1}
: (memref<?xf32>, index, index) -> f32
%4 = "std.add"(%3, %2) : (f32, f32) -> f32
"affine.store"(%4, %arg3, %arg4, %arg5) {map = #map1}
: (f32, memref<?xf32>, index, index) -> ()
// Blocks end with a terminator Op.
"affine.terminator"() : () -> ()
// Ops have a list of attributes.
}) {lower_bound = () -> (0), step = 1 : index, upper_bound = #map3}
: (index) -> ()
"affine.terminator"() : () -> ()
}) {lower_bound = () -> (0), step = 1 : index, upper_bound = #map3}
: (index) -> ()

```

图4: 使用仿射和标准方言进行多项式乘法的MLIR *generic*表示。相同的IR在图8中以自定义语法显示。

操作 MLIR中的语义单位是“操作”，称为*Op*。从“指令”到“函数”再到“模块”，在这个系统中都被建模为操作。MLIR没有固定的操作集合，而是允许（并鼓励）用户自定义扩展——编译器通道会保守地处理未知操作，并且MLIR通过特性、特权操作钩子和优化接口（如第6.1节所述）提供了丰富的支持，用于向通道描述操作的语义。

操作（见图3）具有唯一的操作码，在文本上，它是一个标识其方言和操作的字符串。操作接受并产生零个或多个*values*，分别称为*operands*和*results*，这些都以SSA形式维护。所有值都有一个类型，类似于LLVM IR。除了操作码、操作数和结果外，操作还可能具有*Attributes*、*Regions*、*Block Arguments*和*Location Information*。图4展示了值和操作，%-标识符是（包的）命名值，如果多于一个，则用“:”指定包中的数量，用“#”指定特定值。在通用文本表示中，操作名是带引号的字符串字面量，后跟括号内的操作数。

属性 MLIR属性是结构化的编译时静态信息，例如整型常数值、字符串数据或常量浮点值列表。属性具有类型，每个Op实例都有一个从字符串名称到属性值的开放式键值字典。在通用语法中，属性位于Op操作数与其类型之间，以大括号括起的逗号分隔键值对列表形式呈现。例如，图4使用属性来定义已知为常量仿射形式的循环边界： {lower\_bound = () -> (0), step = 1 : index, upper\_bound = #map3} 其中lower\_bound、upper\_bound和step是属性名称。( ) -> (0)符号用于内联仿射形式，在此例中生成一个产生常量0值的仿射函数。#map3符号用于属性*aliases*，它允许预先将属性值与标签关联，并在需要属性值的任何地方使用该标签。

与操作码一样，属性也没有固定的集合。属性的含义要么来自操作语义，要么来自它们所关联的方言（第3节）。属性也是可扩展的，允许直接引用外部数据结构，这对于与现有系统集成非常有用。

systems. For example, an attribute may refer to the contents of (known at compile time) data storage in an ML system.

**Location information** MLIR provides a compact representation for *location information*, and encourages the processing and propagation of this information throughout the system. It can be used to keep the source program stack trace that produced an Op, to generate debug information. It standardizes the way to emit diagnostics from the compiler, and is used by a wide range of testing tools.

Location information is also extensible, allowing a compiler to refer to existing location tracking systems, high-level AST nodes, LLVM-style file-line-column address, DWARF debugging info or whatever else is needed for a high quality implementation.

**Regions and blocks** An instance of an Op may have a list of attached regions. A *region* provides the mechanism for nested structure in MLIR: a region contains a list of blocks, and a block contains a list of operations (which may contain regions). As with attributes, the semantics of a region are defined by the operation they are attached to, however the blocks inside the region (if more than one) form a Control Flow Graph (CFG). For example, the `affine.for` operation in Figure 4 is a loop with the single-block body attached as a region, located between (`{` and `}`) delimiters. The Op specifies the flow of control across regions. In this example, the body is executed repeatedly until the upper bound is reached.

The body of each region is a list of *blocks*, and each block ends with a *terminator* operation, that may have *successor* blocks to which the control flow may be transferred. Each terminator (e.g. “switch”, “conditional branch” or “unwind”) defines its own semantics. It may chose to transfer the control flow to another block in the same region, or return it to the Op enclosing the region. The graph of successors defines a CFG, allowing standard SSA-based control flow within a region.

Instead of using  $\phi$  nodes, MLIR uses a functional form of SSA [2] where terminators pass values into *block arguments* defined by the successor block. Each block has a (potentially empty) list of typed block arguments, which are regular values and obey SSA. The semantics of terminator Ops defines what values the arguments of the block will take after the control is transferred. For the first (entry) block of the region, the values are defined by the semantics of the enclosing Op. For example, `affine.for` uses the entry block argument `%arg4` as loop induction variable.

**Value dominance and visibility** Ops can only use values that are in scope, i.e. *visible* according to SSA dominance, nesting, and semantic restrictions imposed by enclosing operations. Values are visible within a CFG if they obey standard SSA dominance relationships, where control is guaranteed to pass through a definition before reaching a use.

Region-based visibility is defined based on simple nesting of regions: if the operand to an Op is outside the current region, then it must be defined lexically above and outside the region of the use. This is what allows Ops within an `affine.for` operation to use values defined in outer scopes.

MLIR also allows operations to be defined as *isolated from above*, indicating that the operation is a scope barrier—e.g. the “std.func” Op defines a function, and it is not valid for operations within the function to refer to values defined outside the function. In addition to providing useful semantic checking, a module containing isolated-from-above Ops may be processed in parallel by an MLIR compiler since no use-def chains may cross the isolation barriers. This is a important for compilation to utilize multicore machines.

**Symbols and symbol tables** Ops can have a symbol table attached. This table is a standardized way of associating names, represented as strings, to IR objects, called *symbols*. The IR does not prescribe what symbols are used for, leaving it up to the Op definition. Symbols are most useful for named entities need not obey SSA: they cannot be redefined within the same table, but they can be used prior to their definition. For example, global variables, functions or named modules can be represented as symbols. Without this mechanism, it would have been impossible to define, e.g., recursive function referring to themselves in their definition. Symbol tables can be nested if an Op with a symbol table attached has associated regions containing similar Ops. MLIR provides a mechanism to reference symbols from an Op, including nested symbols.

系统。例如，一个属性可能指代ML系统中（在编译时已知的）数据存储的内容。

位置信息 MLIR 为*location information*,提供了紧凑的表示形式，并鼓励在整个系统中处理和传播这些信息。它可用于保留产生Op的源程序堆栈跟踪，以生成调试信息。它标准化了编译器发出诊断信息的方式，并被广泛的测试工具所使用。

位置信息也是可扩展的，允许编译器引用现有的位置跟踪系统、高级AST节点、LLVM风格的文件-行-列地址、DWARF调试信息或高质量实现所需的任何其他内容。

区域与块 一Op实例可能附带一系列区域。*region*为MLIR中的嵌套结构提供了机制：一个区域包含一系列块，而一个块包含一系列操作（这些操作可能包含区域）。与属性类似，区域的语义由其所属的操作定义，但区域内的块（如果不止一个）会形成一个控制流图（CFG）。例如，图4中的`affine.for`操作是一个循环，其单块主体作为一个区域附加，位于（`{`和`}`）分隔符之间。该Op指定了跨区域的控制流。在此示例中，主体会重复执行，直到达到上限。

每个区域的主体是一个*blocks*列表，每个块以*terminator*操作结束，该操作可能包含*successor*个块，控制流可能转移到这些块。每个终止符（例如“switch”、“条件分支”或“unwind”）定义其自身的语义。它可以选择将控制流转移到同一区域内的另一个块，或将其返回到包含该区域的Op。后继图定义了一个CFG，允许在区域内进行基于SSA的标准控制流。

MLIR采用了一种函数形式的SSA[2]，而非使用 $\phi$ 节点，其中终结指令将值传递给由后继块定义的*block arguments*。每个块都有一个（可能为空的）类型化块参数列表，这些参数是常规值并遵循SSA。终结指令操作的语义定义了控制转移后块参数将接收的值。对于区域的第一个（入口）块，其值由外层操作的语义定义。例如，`affine.for`使用入口块参数`%arg4`作为循环归纳变量。

值支配与可见性操作只能使用作用域内的值，即根据SSA支配关系、嵌套结构以及外层操作施加的语义限制所确定的*visible*值。在控制流图中，若值遵循标准SSA支配关系——即控制流在到达使用点前必然经过其定义点——则该值在图中可见。

基于区域的可见性是根据区域的简单嵌套来定义的：如果一个操作的操作数在当前区域之外，那么它必须在词法上定义于使用区域的上方和外部。这使得`affine.for`操作内的操作能够使用外部作用域中定义的值。

MLIR 还允许将操作定义为 *isolated from above*，表示该操作是一个作用域屏障——例如，“std.func”操作定义了一个函数，函数内部的操作引用函数外部定义的值是无效的。除了提供有用的语义检查外，包含隔离上层操作的模块可以由MLIR编译器并行处理，因为使用-定义链不能跨越隔离屏障。这对于利用多核机器进行编译非常重要。

符号与符号表 操作Op可以附带一个符号表。该表是一种标准化方法，用于将字符串表示的名称与称为*symbols*的IR对象关联起来。IR并未规定符号的具体用途，而是交由操作定义自行决定。符号对于无需遵循SSA的命名实体最为有用：它们在同一表中不能被重新定义，但可以在定义之前使用。例如，全局变量、函数或命名模块可以表示为符号。若无此机制，将无法定义诸如在自身定义中引用自身的递归函数等结构。如果一个附带符号表的操作包含具有类似操作的关联区域，则符号表可以嵌套。MLIR提供了一种从操作中引用符号（包括嵌套符号）的机制。

**Dialects** MLIR manages extensibility using *Dialects*, which provide a logical grouping of Ops, attributes and types under a unique namespace. Dialects themselves do not introduce any new semantics but serve as a logical grouping mechanism and can be used to provide dialect generic Op support (e.g., constant folding behavior for all ops in the dialect). The dialect namespace appears as a dot-separated prefix in the opcode, e.g., Figure 4 uses `affine` and `std` dialects.

The separation of Ops, types and attributes into dialects is conceptual and is akin to designing a set of modular libraries. For example, a dialect can contain Ops and types for operating on hardware vectors (e.g., shuffle, insert/extract element, mask), and another dialect can contain Ops and types for operating on algebraic vectors (e.g. absolute value, dot product, etc.). Whether both dialects use the same vector type and where does this type belong are design decisions left to the user of MLIR.

While it is possible to put all Ops, types and attributes in a single dialect, it would quickly become unmanageable due to the large number of simultaneously present concepts and name conflicts, amongst other issues. Although each Op, type and attribute belongs to exactly one dialect, MLIR explicitly supports a mix of dialects to enable progressive lowering. Ops from different dialects can coexist at any level of the IR at any time, they can use types defined in different dialects, etc. Intermixing of dialects allows for greater reuse, extensibility and provides flexibility that otherwise would require developers to resort to all kinds of non-composable workarounds.

**Type system** Each value in MLIR has a type, which is specified in the Op that produces the value or in the block that defines the value as an argument. Types provide compile-time semantics for the IR. The type system in MLIR is user-extensible, and may refer to existing foreign type systems (e.g. an `llvm::Type` or a `clang::Type`). MLIR enforces strict type equality checking and does not provide type conversion rules. Ops list their inputs and result types using trailing function-like syntax. In Figure 4, `std.load` maps from the memory reference and index types to the type of the value it loads.

From the type theory point of view, MLIR only supports non-dependent types, including trivial, parametric, function, sum and product types. While it is possible to implement a dependent type system by combining Ops with symbols and user-defined types in a literal interpretation of Curry-Howard isomorphism, such types will be opaque to the IR.

**Standard types** In addition, MLIR provides a standardized set of commonly used types, including arbitrary precision integers, standard floating point types, and simple common containers—tuples, multi-dimensional vectors, and tensors. These types are merely a convenience that are useful to authors of dialects, but their use is not required.

**Functions and modules** Similarly to conventional IRs, MLIR is usually structured into functions and modules. However, these are not new or separate concepts in MLIR: they are implemented as Ops in the builtin dialect.

A module is an Op with a single region containing a single block, and terminated by a dummy Op that does not transfer the control flow. A module defines a symbol and can be referenced. Like any block, its body contains a list of Ops, which may be functions, global variables, compiler metadata, or other top-level constructs.

A function is an Op with a single region, with arguments corresponding to function arguments. It defines a symbol and can be referenced by name. The control flow is transferred into a function using a function call Op. Once inside, the control flow follows the CFG of the blocks in the region. A “return” terminator does not have successors and instead terminates the region execution, transferring the control flow back to the call-site of the function. Any operands of the “return” terminator Op are the returned values of the function.

## 4 IR Infrastructure

Beyond the IR itself, MLIR provides infrastructure for defining IR elements such as dialects, Ops, pattern rewrites, verification and reusable passes. The infrastructure aspect of MLIR is essential for providing extensibility and ease of use when defining new abstractions and using MLIR as an optimization toolkit.

方言 MLIR 通过*Dialects*管理可扩展性，它在唯一的命名空间下对操作、属性和类型进行逻辑分组。方言本身并不引入新的语义，而是作为一种逻辑分组机制，可用于提供方言通用的操作支持（例如，方言中所有操作的常量折叠行为）。方言命名空间在操作码中以点分隔的前缀形式出现，例如，图4使用了仿射和标准方言。

将操作、类型和属性分离到不同的方言中是一种概念上的划分，类似于设计一套模块化库。例如，一个方言可以包含用于操作硬件向量的操作和类型（如洗牌、插入/提取元素、掩码等），而另一个方言可以包含用于操作代数向量的操作和类型（如绝对值、点积等）。是否两个方言使用相同的向量类型，以及这种类型归属于哪个方言，这些设计决策都留给MLIR的用户来决定。

虽然可以将所有操作、类型和属性放在一个方言中，但由于同时存在的概念数量庞大、名称冲突等问题，这很快就会变得难以管理。尽管每个操作、类型和属性都严格属于一个方言，但MLIR明确支持多种方言混合，以实现渐进式降级。来自不同方言的操作可以在IR的任何层级随时共存，它们可以使用不同方言中定义的类型等。方言的混合允许更大的重用性和可扩展性，并提供了灵活性，否则开发者将不得不诉诸各种不可组合的变通方法。

**类型系统** MLIR中的每个值都有一个类型，该类型在产生该值的操作中指定，或在定义该值作为参数的块中指定。类型为IR提供编译时语义。MLIR中的类型系统可由用户扩展，并可引用现有的外部类型系统（例如`llvm::Type`或`clang::Type`）。MLIR强制执行严格的类型相等性检查，不提供类型转换规则。操作使用类似函数的尾随语法列出其输入和结果类型。在图4中，`std.load`从内存引用和索引类型映射到其加载值的类型。

从类型理论的角度来看，MLIR仅支持非依赖类型，包括平凡类型、参数化类型、函数类型、和类型与积类型。虽然通过将操作与符号及用户定义类型结合，并依据柯里-霍华德同构的字面解释来实现依赖类型系统是可能的，但这类类型对中间表示层将是不透明的。

**标准类型** 此外MLIR 提供了一套标准化的常用类型，包括任意精度整数、标准浮点类型以及简单的常见容器——元组、多维向量和张量。这些类型仅为方言编写者提供便利，并非强制要求使用。

**函数与模块** 与传统的中间表示类似MLIR通常被组织为函数和模块。然而，这些在MLIR中并非全新或独立的概念：它们是作为内置方言中的操作（Ops）实现的。

模块是一种操作，它包含一个单一区域，该区域内有一个单一的基本块，并以一个不传递控制流的虚拟操作结束。模块定义了一个符号，并且可以被引用。与任何基本块一样，其主体包含一系列操作，这些操作可以是函数、全局变量、编译器元数据或其他顶层结构。

函数是一种具有单个区域的Op，其参数对应于函数参数。它定义了一个符号，并可通过名称引用。控制流通过函数调用Op转移到函数内部。一旦进入，控制流遵循区域内块的控制流图（CFG）。“返回”终止符没有后继，而是终止区域执行，将控制流转移回函数的调用点。“返回”终止符Op的任何操作数都是函数的返回值。

## 4 IR基础设施

除了IR本身，MLIR还提供了定义IR元素的基础设施，例如方言、操作、模式重写、验证和可重用通道。MLIR的基础设施方面对于在定义新抽象以及将MLIR用作优化工具包时提供可扩展性和易用性至关重要。



```

// An Op is a TableGen definition that inherits the "Op" class parameterized
// with the Op name
def LeakyReluOp: Op<"leaky_relu",
  // and a list of traits used for verification and optimization.
  [NoSideEffect, SameOperandsAndResultType]> {
  // The body of the definition contains named fields for a one-line
  // documentation summary for the Op.
  let summary = "Leaky Relu operator";

  // The Op can also a full-text description that can be used to generate
  // documentation for the dialect.
  let description = [{
    Element-wise Leaky ReLU operator
    x -> x >= 0 ? x : (alpha * x)
  }];

  // Op can have a list of named arguments, which include typed operands
  // and attributes.
  let arguments = (ins AnyTensor:$input, F32Attr:$alpha);

  // And a list of named and typed outputs.
  let results = (outs AnyTensor:$output);
}

```

Figure 5: Operation Definition Syntax (ODS) provides a concise way of defining new Ops in MLIR. Here, one defines the LeakyRelu Op taking a tensor and a floating-point value, and returning a tensor of the same type as the input one.

#### 4.1 Operation description

MLIR uses TableGen-based [47] specification for Operation Descriptions (ODS), defining the structure of an Op and components of its verifier declaratively. TableGen is a data modeling tool intended to help define and maintain records of domain-specific information, used extensively in LLVM. We chose it for modeling Ops and rewrite patterns to leverage its acceptance by the industry. ODS can be seen as a DSL for MLIR Op definition *embedded* into the TableGen input language, so the ODS syntax is imposed by TableGen, but the MLIR-specific semantics is provided by ODS. The ODS definition is ultimately translated into C++ code (including Op classes with named accessors, verification, etc.) which interoperate with the rest of the system.

Ops are modeled in ODS using the TableGen Op class. Figure 5 shows an example of Op ODS definition. Each defined Op has a *name* which is a unique identifier, a list of *traits* that describe Op properties, a list of *arguments* that specify Op’s operands and attributes, and a list of Op’s *results*. Arguments and results have names and type constraints (e.g., fixed-shape tensor of float or int32). Op definition may also specify human readable Op description for the documentation. And a (limited) custom textual form for which custom printer/parser will be generated. When Op definition requires finer-grain control than ODS provides, additional C++ code can be injected via *builder*, *printer*, *parser*, *verifier* clauses. Op traits can be generic, e.g., “has no side-effects”, and dialect- or ODS-specific, e.g., “has custom exporter”. Traits in ODS may be backed by C++ classes defining the behavior of the trait. There is no fixed set of traits, but some traits are known by ODS (e.g., “shape result and operand type” represents a constraint that fully captures the output type given input types) or optimizers (e.g., “has no side-effects”, see Section 6.1).

Type constraints check properties of the type of arguments/results and are user/dialect extensible. MLIR infrastructure also provides numerous pre-defined type constraints, such as “any type”, “tensor with element satisfying the given constraint”, “vector of given rank”, etc. ODS also has limited support for automatically deducing the return type of results of operands using the constraints induced by the traits, see Section 4.2 for more information.

#### 4.2 Declarative rewrites

Many MLIR transformations involve Op manipulations, and while some transformations require complex modifications of the IR, many others can be expressed as simple rewrites on the DAG

```

// An Op is a TableGen definition that inherits the "Op" class parameterized
// with the Op name
def LeakyReluOp: Op<"leaky_relu",
  // and a list of traits used for verification and optimization.
  [NoSideEffect, SameOperandsAndResultType]> {
  // The body of the definition contains named fields for a one-line
  // documentation summary for the Op.
  let summary = "Leaky Relu operator";

  // The Op can also a full-text description that can be used to generate
  // documentation for the dialect.
  let description = [{
    Element-wise Leaky ReLU operator
    x -> x >= 0 ? x : (alpha * x)
  }];

  // Op can have a list of named arguments, which include typed operands
  // and attributes.
  let arguments = (ins AnyTensor:$input, F32Attr:$alpha);

  // And a list of named and typed outputs.
  let results = (outs AnyTensor:$output);
}

```

图5: 操作定义语法（ODS）提供了一种在MLIR中定义新Op的简洁方式。此处定义了一个LeakyRelu Op，它接收一个张量和一个浮点值，并返回一个与输入张量类型相同的张量。

#### 4.1 操作说明

MLIR采用基于TableGen[47]的操作描述规范（ODS），以声明式方式定义Op的结构及其验证器组件。TableGen是一种数据建模工具，旨在帮助定义和维护领域特定信息的记录，在LLVM中被广泛使用。我们选择它来建模Op和重写模式，以利用其在行业中的接受度。ODS可视为将MLIR Op定义*embedded*转换为TableGen输入语言的领域特定语言，因此ODS语法由TableGen规定，但MLIR特定的语义由ODS提供。ODS定义最终会被翻译成C++代码（包括具有命名访问器、验证等功能的Op类），这些代码与系统的其余部分互操作。

操作（Op）在ODS中使用TableGen的Op类进行建模。图5展示了一个Op ODS定义的示例。每个已定义的操作都有一个唯一的标识符*name*，一个描述Op属性的*traits*列表，一个指定Op操作数和属性的*arguments*列表，以及Op的*results*列表。参数和结果具有名称和类型约束（例如，固定形状的浮点或int32张量）。Op定义还可以为文档指定人类可读的Op描述，以及一个（有限的）自定义文本格式，用于生成自定义的打印机/解析器。当Op定义需要比ODS提供的更细粒度控制时，可以通过*builder*、*printer*、*parser*、*verifier*子句注入额外的C++代码。Op特性可以是通用的，例如“无副作用”，也可以是特定于方言或ODS的，例如“具有自定义导出器”。ODS中的特性可能由定义特性行为的C++类支持。特性没有固定集合，但某些特性是ODS已知的（例如，“形状结果和操作数类型”表示一个约束，完全捕获给定输入类型的输出类型）或优化器已知的（例如“无副作用”，参见第6.1节）。

类型约束检查参数/结果的类型属性，并且可由用户/方言扩展。MLIR基础设施还提供了许多预定义的类型约束，例如“任意类型”、“元素满足给定约束的张量”、“给定秩的向量”等。ODS还支持通过特性诱导的约束自动推导操作数结果的返回类型，更多信息请参见第4.2节。

#### 4.2 声明式重写

许多MLIR转换涉及操作（Op）的操纵，虽然一些转换需要对IR进行复杂的修改，但许多其他转换可以表示为DAG上的简单重写。

```
// A rewrite pattern declaring an equivalence between a source and target DAG.
def : Pattern<
  // The first parameter is the source DAG with nodes corresponding to Ops in
  // ODS followed by a list of named arguments, potentially with more type
  // constraints.
  (LeakyReluOp $arg, F32Attr:$alpha),

  // The second parameter is a list of DAGs that is used to replace matched
  // source DAG. DRR supports multiple result patterns as well as auxiliary
  // Ops, which are used for building the replacement ops but are not
  // directly used for replacements. The DAGs may refer to matched values in
  // the source DAG or those created in target.
  [(SelectOp (CmpFOp CMPF_P_ODT
    // A referenced argument bound in the source DAG.
    $arg
    // Nested DAG construct Ops that produce value from attribute created in
    // pattern.
    (ConstantOp ConstantAttr<F32Attr, "0.">))
    // And the two final, required arguments of SelectOp.
    $arg, (ConstantOp $alpha))]>;
```

Figure 6: Declarative graph rewrite rule transforming a LeakyRelu into a Compare-Float, followed by a Select.

defined by SSA use-def relations. MLIR provides a graph rewriting framework complemented with the Declarative Rewrite Rule (DRR) system that makes it simple to express patterns.

Similarly to ODS, DRR is a DSL embedded into the TableGen language. DRR expresses source and target DAG patterns along with constraints (including dynamic constraints [49]) and benefits for pattern prioritization. Patterns can capture and reuse arguments of an Op. Conceptually, DRR expresses equivalence of DAGs under specific constraints. Figure 6 gives an example of a DRR pattern that converts an Op defined in Figure 5 into common lower-level implementation consisting of a `compare` and a `select`.

DRR is converted into C++ code, which can be intermixed with more complex patterns defined directly in C++ using the generic graph rewriting framework. This ability allows MLIR to keep the common case simple without restricting generality of the framework.

### 4.3 Pass manager

The MLIR pass manager organizes and handles the efficient execution of a series of *IR passes* operating on various granularities. Whereas pass management in existing systems is typically defined over a fixed granularity (e.g., module, function or loop pass managers), in MLIR modules and functions are not special—they are merely Ops with regions and there can be multiple variants of them. Therefore, the MLIR pass manager is also not specialized on a fixed set of ops, but instead works on arbitrary Ops at arbitrary levels of nesting.

**Parallel compilation** An important requirement of MLIR is the need to utilize multi-core machines for faster compilation. The pass manager supports concurrent traversal and modification of the intermediate representation, which is made possible by invariants provided by “isolated-from-above” property of operations, because the SSA use-def chains cannot cross the region boundaries of these ops. Operations with this behavior (e.g. the “std.func” operation) thus define a tree of regions that may be processed in parallel.

This requirement is the reason why (in contrast to, for example, LLVM), MLIR does not feature whole-module use-def chains. Global objects are referenced through symbol table entries, and constants are implemented as operations with associated attributes.

### 4.4 Round-trippable textual IR form

The IR and Ops in MLIR have a textual representation that fully reflects the in-memory representation, which is critical for debugging, understanding the IR during transformations, and for writing test

```
// A rewrite pattern declaring an equivalence between a source and target DAG.
def : Pattern<
  // The first parameter is the source DAG with nodes corresponding to Ops in
  // ODS followed by a list of named arguments, potentially with more type
  // constraints.
  (LeakyReluOp $arg, F32Attr:$alpha),

  // The second parameter is a list of DAGs that is used to replace matched
  // source DAG. DRR supports multiple result patterns as well as auxiliary
  // Ops, which are used for building the replacement ops but are not
  // directly used for replacements. The DAGs may refer to matched values in
  // the source DAG or those created in target.
  [(SelectOp (CmpFOp CMPF_P_ODT
    // A referenced argument bound in the source DAG.
    $arg
    // Nested DAG construct Ops that produce value from attribute created in
    // pattern.
    (ConstantOp ConstantAttr<F32Attr, "0.">))
    // And the two final, required arguments of SelectOp.
    $arg, (ConstantOp $alpha))]>;
```

图6: 将LeakyRelu转换为Compare-Float后接Select的声明式图重写规则。

由SSA使用-定义关系定义。MLIR提供了一个图重写框架，并辅以声明式重写规则（DRR）系统，使得表达模式变得简单。

与ODS类似，DRR是一种嵌入TableGen语言的领域特定语言。DRR通过源和目标DAG模式以及约束条件（包括动态约束[49]）来表达，并支持模式优先级的优化。模式能够捕获并重用Op的参数。从概念上讲，DRR表达了特定约束下DAG的等价性。图6展示了一个DRR模式的示例，该模式将图5中定义的Op转换为由比较和选择操作组成的常见低级实现。

DRR被转换为C++代码，可以与直接在C++中使用通用图重写框架定义的更复杂模式混合。这种能力使MLIR能够在保持常见情况简单的同时，不限制框架的通用性。

### 4.3 通道管理器

MLIR 通行证管理器负责组织并高效执行一系列在不同粒度上操作的*IR passes*。现有系统中的通行证管理通常基于固定粒度（例如模块、函数或循环通行证管理器）定义，而在MLIR中，模块和函数并非特殊存在——它们仅是带有区域的运算操作，且可能存在多种变体。因此，MLIR通行证管理器也不针对固定运算操作集进行专门化，而是能在任意嵌套层级上处理任意运算操作。

并行编译 MLIR的一个重要需求是利用多核机器实现更快的编译。通行证管理器支持对中间表示进行并发遍历和修改，这得益于操作的“与上层隔离”特性所提供的约束条件，因为SSA使用-定义链无法跨越这些操作的区域边界。具有此行为的操作（例如“std.func”操作）因此定义了一个可以并行处理的区域树。

这一要求解释了为何（与LLVM等系统不同）MLIR不采用全模块使用-定义链。全局对象通过符号表条目进行引用，而常量则通过附带属性的操作来实现。

### 4.4 可往返的文本IR形式

MLIR中的IR和操作具有完全反映内存中表示的文本表示形式，这对于调试、在转换过程中理解IR以及编写测试至关重要。

cases. The raw IR form shown in Figure 4 is verbose and difficult to understand. Therefore MLIR allows for defining custom printing and parsing formats for Ops, which allows the example to be printed and parsed as shown in Figure 8, which is much easier to work with.

Both forms are fully round-trippable and each compiler pass may be tested separately, using the textual form as both input and output. Since there is no hidden state, the result of running an individual pass is identical to running the same pass in the full pass pipeline. This approach is user-friendly, as the IR form can be created by hand, and IR transformations are easy to trace.

#### 4.5 Documentation

Dialects, Ops and Interfaces have a documentation generated from their ODS descriptions. Beyond one line *summary* and more readable *description*, the generated documentation includes argument and result type constraints. As the same source is used for both the verification code and documentation, the documentation has higher changes of remaining in sync with runtime behavior.

#### 4.6 Verifiers

*Verifiers* are used to enforce structural correctness of the IR and invariants of Ops, allowing passes to assume a verified IR with invariants checked, and also serves as a debugging tool. Verification starts with checking structural properties of MLIR overall: types must match exactly, values are defined only once and obey dominance and visibility, symbol names are unique in the symbol table, all blocks end with terminator Ops, etc. After that, individual Op and attribute verifiers are applied. Each Op may define a set of structural and semantic rules that check its validity. For example, a binary Op checks that it has two operands, many Ops only accept values of specific types, and many require specific attributes or regions attached. Similarly, dialect attributes can be only allowed on specific Ops or impose further restrictions on the Ops to which they are attached. For example, a dialect attribute can require an Op to only use types defined in the dialect even though the Op itself is more generic. Verification failures are considered invariant violations and abort compilation.

### 5 Evaluation: Applications of MLIR

MLIR is a system that aims to generalize and drive a wide range of compiler projects, so our primary evaluation metric is to show that it is being adopted and used for diverse projects. We provide a summary of community activity and describe a few use cases in more detail to highlight the generality and extensibility of MLIR and demonstrate how well it implements the customizability design principle.

Today, MLIR is a growing open source project with a community spanning academia and industry. For example, the academic workshop about use of MLIR in High-Performance Computing (HPC) was attended by individuals from 16 universities and involved 4 national laboratories from 4 different countries. MLIR was also endorsed by 14 multinational companies and at the LLVM Developer Meeting more than 100 industry developers attended a roundtable event about MLIR. Community adoption and participation is a proxy measure for usability and need. More than 26 dialects are in development in public or private and 7 projects across different companies are replacing custom infrastructure with MLIR. We argue that this shows a real need for MLIR, as well as endorses its usability.

#### 5.1 TensorFlow graphs

While the other discussed representations are familiar to most compiler developments, one of key use cases for MLIR is to support the development of machine learning frameworks. Their internal representations is often based on a data flow graph [53] with a dynamic execution semantics.

TensorFlow [1] is an example of such framework. Its representation is a high-level dataflow computation where the nodes are computations which can be placed on various devices, including specific hardware accelerators.

MLIR is used in TensorFlow to model this internal representation and perform transformations for the use cases presented in Figure 1: from simple algebraic optimizations to retargeting graphs for

案例。图4中展示的原始IR形式冗长且难以理解。因此，MLIR允许为操作定义自定义的打印和解析格式，这使得示例能够以图8所示的形式进行打印和解析，大大提升了使用的便捷性。

两种形式都完全支持往返转换，并且每个编译器阶段都可以单独测试，使用文本形式作为输入和输出。由于没有隐藏状态，运行单个阶段的结果与在完整阶段流水线中运行同一阶段完全相同。这种方法对用户友好，因为IR形式可以手动创建，且IR转换易于追踪。

#### 4.5 文档

方言、操作和接口的文档是从它们的ODS描述生成的。除了单行*summary*和更易读的*description*之外，生成的文档还包括参数和结果类型的约束。由于验证代码和文档使用相同的来源，文档更有可能与运行时行为保持同步。

#### 4.6 验证器

*Verifiers* 用于确保IR的结构正确性和Op的不变性，允许传递过程假设一个已验证且不变性已检查的IR，同时也作为调试工具。验证从检查MLIR的整体结构属性开始：类型必须完全匹配，值仅定义一次并遵循支配性和可见性规则，符号表中符号名称唯一，所有块以终结符Op结束等。之后，应用单个Op和属性的验证器。每个Op可以定义一组检查其有效性的结构和语义规则。例如，二元Op检查其有两个操作数，许多Op仅接受特定类型的值，许多要求附加特定属性或区域。类似地，方言属性可能仅允许在特定Op上使用，或对附加它们的Op施加进一步限制。例如，方言属性可以要求Op仅使用该方言中定义的类型，即使Op本身更为通用。验证失败被视为不变性违反，并中止编译。

### 5 评估：MLIR的应用

MLIR是一个旨在泛化并推动广泛编译器项目的系统，因此我们的主要评估指标是展示其正被多样化的项目采纳和使用。我们总结了社区活动，并详细描述了几个用例，以突显MLIR的通用性和可扩展性，并展示其在实现可定制化设计原则方面的卓越表现。

如今，MLIR已成为一个不断发展的开源项目，其社区横跨学术界与工业界。例如，关于MLIR在高性能计算（HPC）中应用的学术研讨会吸引了来自16所高校的研究者，并涉及4个不同国家的4个国家实验室。MLIR还获得了14家跨国公司的支持，在LLVM开发者大会上，超过100名工业界开发者参与了关于MLIR的圆桌讨论。社区的采纳与参与度是衡量其可用性和需求的重要指标。目前有超过26种方言在公开或私有领域开发中，7家不同公司的项目正在用MLIR替代其定制基础设施。我们认为这充分证明了市场对MLIR的真实需求，也印证了其可用性。

#### 5.1 TensorFlow 图

虽然其他讨论的表示形式对大多数编译器开发者来说都很熟悉，但MLIR的一个关键用例是支持机器学习框架的开发。这些框架的内部表示通常基于具有动态执行语义的数据流图[53]。

TensorFlow [1] 便是此类框架的一个例子。其表现形式是一种高层次的数据流计算，其中节点是可放置于包括特定硬件加速器在内的多种设备上的计算单元。

MLIR在TensorFlow中被用来建模这种内部表示，并针对图1中展示的用例执行转换：从简单的代数优化到重定向图。



```

%0 = tf.graph (%arg0 : tensor<f32>, %arg1 : tensor<f32>,
               %arg2 : !tf.resource) {
  // Execution of these operations is asynchronous, the %control return value
  // can be used to impose extra runtime ordering, for example the assignment
  // to the variable %arg2 is ordered after the read explicitly below.
  %1, %control = tf.ReadVariableOp(%arg2)
    : (!tf.resource) -> (tensor<f32>, !tf.control)
  %2, %control_1 = tf.Add(%arg0, %1)
    : (tensor<f32>, tensor<f32>) -> (tensor<f32>, !tf.control)
  %control_2 = tf.AssignVariableOp(%arg2, %arg0, %control)
    : (!tf.resource, tensor<f32>) -> !tf.control
  %3, %control_3 = tf.Add(%2, %arg1)
    : (tensor<f32>, tensor<f32>) -> (tensor<f32>, !tf.control)
  tf.fetch %3, %control_2 : tensor<f32>, !tf.control
}

```

Figure 7: SSA representation of a TensorFlow dataflow graph in MLIR.

parallel execution on data center clusters of hardware accelerators, from lowering to a representation suitable for mobile deployment to generating efficient native code using tools like XLA [57]. The representation of a TensorFlow graph in MLIR is illustrated on Figure 7.

## 5.2 Polyhedral code generation

One of the original motivations for MLIR was the exploration of polyhedral code generation for accelerators. The affine dialect is a simplified polyhedral representation that was designed to enable progressive lowering. While a full exploration of the design points here is out of scope for this paper, we illustrate aspects of the affine dialect to show the modeling power of MLIR and contrast the affine dialect with past representations [17, 19, 54, 55, 52].

### 5.2.1 Similarities

The MLIR affine dialect operates on a structured multi-dimensional type for all accesses to memory. In the default case, these structured types are *injective*: different indexings are guaranteed not to alias by construction, a common precondition for polyhedral dependence analyses.

Affine modeling is split in two parts. Attributes are used to model affine maps and integer sets at compile-time and Ops are used to apply affine restrictions to the code. Namely, `affine.for` Op is a “for” loop with bounds expressed as affine maps of values required to be invariant in a function. Thus loops have static control flow. Similarly, `affine.if` is a conditional restricted by affine integer sets. The bodies of loops and conditionals are regions that use `affine.load` and `affine.store` to restrict indexing to affine forms of surrounding loop iterators. This enables exact affine dependence analysis while avoiding the need to infer affine forms from a lossy lower-level representation.

### 5.2.2 Differences

The differences with existing polyhedral frameworks are numerous, we can characterize them in four categories:

(1) *Rich types*: the MLIR structured memory reference type contains a layout map connecting the index space of the buffer to the actual address space. This separation of concerns makes loop and data transformations compose better: changes to data layout do not affect the code and do not pollute dependence analysis. Such mixes of transformations have been explored previously [38] but are uncommon.

(2) *Mix of abstractions*: Bodies of affine loops in MLIR can be expressed with operations on typed SSA values. Therefore, all traditional compiler analyses and transformations remain applicable and can be interleaved with polyhedral transformations. On the contrary, polyhedral compilers often abstract such details away completely, making it challenging for a polyhedral compiler to manipulate, e.g., vector types.

```

%0 = tf.graph (%arg0 : 张量<f32>, %arg1 : 张量<f32>,          %arg2 : !tf.resource) { // 这些操作的执行是异步的，返回值 %control // 可用于施加额外的运行时顺序，例如对变量 %arg2 的赋值 // 被显式安排在下方读取操作之后。
  %1, %control = tf.ReadVariableOp(%arg2)    : (!tf.resource) -> (张量<f32>, !tf.control)
  %2, %control_1 = tf.Add(%arg0, %1)         : (张量<f32>, 张量<f32>) -> (张量<f32>, !tf.control)
  %control_2 = tf.AssignVariableOp(%arg2, %arg0, %control) : (!tf.resource, 张量<f32>) -> !tf.control
  %3, %control_3 = tf.Add(%2, %arg1)         : (张量<f32>, 张量<f32>) -> (张量<f32>, !tf.control)
  tf.fetch %3, %control_2 : 张量<f32>, !tf.con
}

```

图7：MLIR中TensorFlow数据流图的SSA表示。

在数据中心集群的硬件加速器上进行并行执行，从降低到适合移动部署的表示形式，到使用XLA等工具生成高效的本机代码[57]。图7展示了MLIR中TensorFlow图的表示。

## 5.2 多面体代码生成

MLIR最初的动机之一是探索面向加速器的多面体代码生成。仿射方言是一种简化的多面体表示形式，旨在实现渐进式降级。虽然全面探讨此处的设计要点超出了本文范围，但我们通过展示仿射方言的多个方面来体现MLIR的建模能力，并将仿射方言与过往的表示形式进行对比[17, 19, 54, 55, 52]。

### 5.2.1 相似之处

MLIR仿射方言对所有内存访问操作都使用结构化的多维类型。在默认情况下，这些结构化类型为*injective*：通过构造保证不同的索引不会产生别名，这是多面体依赖分析的一个常见前提条件。

仿射建模分为两部分。属性用于在编译时建模仿射映射和整数集，而操作（Ops）用于将仿射约束应用于代码。具体来说，`affine.for` 操作是一个“for”循环，其边界表示为函数中必须保持不变的值的仿射映射，因此循环具有静态控制流。类似地，`affine.if` 是一个受仿射整数集限制的条件语句。循环和条件语句的主体是使用 `affine.load` 和 `affine.store` 的区域，这些操作将索引限制为周围循环迭代器的仿射形式。这使得能够进行精确的仿射依赖分析，同时避免了从有损的低级表示中推断仿射形式的需要。

### 5.2.2 差异

与现有多面体框架的差异众多，我们可以将其归纳为四大类：

(1) *Rich types*：MLIR结构化内存引用类型包含一个连接缓冲区索引空间与实际地址空间的布局映射。这种关注点分离使得循环与数据变换能更好地组合：数据布局的更改不会影响代码，也不会污染依赖分析。此类变换的混合先前已有研究[38]，但并不常见。

(2) *Mix of abstractions*: MLIR中的仿射循环体可以通过对类型化SSA值的操作来表达。因此，所有传统的编译器分析和转换仍然适用，并且可以与多面体转换交错进行。相反，多面体编译器通常完全抽象掉这些细节，这使得多面体编译器难以操作，例如向量类型。

```

// Affine loops are Ops with regions.
affine.for %arg0 = 0 to %N {
  // Only loop-invariant values, loop iterators, and affine functions of
  // those are allowed.
  affine.for %arg1 = 0 to %N {
    // Body of affine for loops obey SSA.
    %0 = affine.load %A[%arg0] : memref<? x f32>
    // Structured memory reference (memref) type can have
    // affine layout maps.
    %1 = affine.load %B[%arg1] : memref<? x f32, (d0)[s0] -> (d0 + s0)>
    %2 = mulf %0, %1 : f32
    // Affine load/store can have affine expressions as subscripts.
    %3 = affine.load %C[%arg0 + %arg1] : memref<? x f32>
    %4 = addf %3, %2 : f32
    affine.store %4, %C[%arg0 + %arg1] : memref<? x f32>
  }
}

```

Figure 8: Representing polynomial multiplication kernel  $C(i+j) += A(i) * B(j)$  using MLIR affine dialect.

(3) *Smaller representation gap*: One of the key features of the polyhedral model is its ability to represent the order of loop iterations in the *type system*. In this system, a large number of loop transformations compose directly and can be reasoned about using simple mathematical abstractions [19]. However, polyhedral transformations require *raising* into a representation often drastically different from the original [20, 10]. Furthermore, the conversion from transformed polyhedra to loops is computationally hard [7]. MLIR-based representation maintains high-level loop structure around lower-level representation, removing the need for raising.

(4) Compilation speed is a crucial goal for MLIR as discussed in Section 4.3, but has not been a focus of most existing polyhedral approaches. These rely heavily on algorithms with exponential complexity: on integer linear programming to derive loop orderings automatically and on polyhedron scanning algorithms to convert the representation back to loops. The approach taken by MLIR explicitly does not rely on polyhedron scanning since loops are preserved in the IR.

Experience with the affine dialect shows that it is useful for a wide range of code generation projects, and its development was important exploration that the MLIR design made practical.

### 5.3 Fortran IR (FIR)

The LLVM Fortran frontend “flang” is currently under major development, led by NVIDIA/PGI. Similar to Swift, Rust, and others, flang needs a specialized IR in order to support advanced transformations for high-performance Fortran codebase, and is using MLIR to support these Fortran-specific optimizations [44]. These high-level optimizations—advanced loop optimizations, array copy elimination, call specialization, devirtualization—would be hard implement using only LLVM.

For example, FIR is able to model Fortran virtual dispatch table as a first class concept (see Figure 9).

```

// Dispatch table for type(u)
fir.dispatch_table @dtable_type_u {
  fir.dt_entry "method", @u_method
}

func @some_func() {
  %uv = fir.alloca !fir.type<u> : !fir.ref<!fir.type<u>>
  fir.dispatch "method"(%uv) : (!fir.ref<!fir.type<u>>) -> ()
  // ...
}

```

Figure 9: FIR has first class support for dynamic virtual function dispatch table.

```

// 仿射循环是带有区域的运算。affine.for %arg0 = 0 到 %N { // 仅允许循环不变值、循环迭代器及其
// 仿射函数。  affine.for %arg1 = 0 到 %N { // 仿射循环体遵循SSA原则。  %0 = affine.load %A[%ar
// 结构化内存引用 (memref) 类型可包含仿射布局映射。  %1 = affine.load
%B[%arg1] : memref<? x f32, (d0)[s0] -> (d0 + s0)>  %2 = mulf %0, %1 : f32  // 仿射加载/存储可使
用仿射表达式作为下标。  %3 = affine.load %C[%arg0 + %arg1] : memref<? x f32>  %4 = addf %3,
%2 : f32  affine.store %4, %C[%arg0 + %arg1] : memref<? x f32>  }}

```

图8：使用MLIR仿射方言表示多项式乘法核 $C(i+j) += A(i) * B(j)$ 。

(3) *Smaller representation gap*: 多面体模型的关键特性之一是其能够在*type system*中表示循环迭代的顺序。在该系统中，大量循环变换可直接组合，并可通过简单的数学抽象进行推理 [19]。然而，多面体变换要求将*raising*转换为通常与原始表示截然不同的形式[20, 10]。此外，从变换后的多面体到循环的转换在计算上是困难的[7]。基于MLIR的表示在底层表示周围保持了高级循环结构，从而无需进行提升转换。

(4) 如第4.3节所述，编译速度是MLIR的关键目标，但现有大多数多面体方法并未将其作为重点。这些方法严重依赖具有指数级复杂度的算法：一是通过整数线性规划自动推导循环顺序，二是通过多面体扫描算法将表示转换回循环。MLIR采用的方法明确不依赖多面体扫描，因为循环在中间表示中得以保留。

与仿射方言的经验表明，它对广泛的代码生成项目都很有用，其发展是MLIR设计使之成为可能的重要探索。

### 5.3 Fortran IR (FIR)

LLVM的Fortran前端“flang”目前正处于由NVIDIA/PGI主导的重大开发阶段。与Swift、Rust等语言类似，flang需要一个专门的中间表示（IR）来支持高性能Fortran代码库的高级转换，并正在使用MLIR来实现这些Fortran特有的优化[44]。这些高级优化——包括复杂的循环优化、数组拷贝消除、调用特化和去虚拟化——若仅依赖LLVM将难以实现。

例如，FIR能够将Fortran虚拟调度表建模为一等概念（见图9）。

```

// 类型(u)的分派表 fir.dispatch_table @dtable_type_u { fir.dt_entry "method", @u_meth
od }函数 @some_func() { %uv = fir.alloca !fir.type<u> : !fir.ref<!fir.type<u>>  fir.dispa
tch "method"(%uv) : (!fir.ref<!fir.type<u>>) -> ()  // ... }

```

图9：FIR对动态虚函数调度表提供了一流的支持。



The ability to model the high-level semantics of the programming language in a structured IR is very powerful. For example, first-class modeling of the dispatch tables allows a robust devirtualization pass to be implemented. While this could have been implemented with a bespoke compiler IR, the use of MLIR allowed the flang developers to spend their engineering resources focused on the IR design for their domain instead of reimplementing basic infrastructure.

The choice of MLIR also unlocks the reusability of other dialects that are not specific to Fortran: a language-independent OpenMP dialect could be shared between Fortran and C language frontends. Similarly, targeting a heterogeneous platform using OpenACC becomes tractable within MLIR through the sharing and reuse of the GPU-oriented dialects and passes. This is straightforward thanks to MLIR begin specifically designed to support a mix of composable dialects.

### 5.4 Domain-specific compilers

The applications of MLIR above are within large compilation workflows, but it is also used to build domain specific compilers for specific small workflows. A reusable and modular infrastructure makes these specialized paths feasible and relatively cheap to build.

**Optimizing MLIR pattern rewriting** MLIR has an extensible system for pattern rewrites described in Section 4. In addition to statically declared patterns, we had applications where the rewrite patterns needed to be dynamically extensible at runtime, allowing hardware vendors to add new lowerings in drivers. The solution was to express MLIR pattern rewrites as an MLIR dialect itself, allowing us to use MLIR infrastructure to build and optimize efficient Finite State Machine (FSM) matcher and rewriters on the fly. This work includes FSM optimizations seen in other systems, such as the LLVM SelectionDAG and GlobalISel instruction selection systems.

**Lattice regression compiler** Lattice regression [18] is a machine learning technique renowned for fast evaluation times and interpretability. The predecessor of the compiler was implemented using C++ templates. This allowed for high-performance code with metaprogramming, but expressing general optimizations on the end-to-end models was not straightforward. This particular lattice regression system is used in applications with multiple millions of users and hence performance improvements are critical.

MLIR was used as the basis for a new compiler for this specialized area, which was driven by a specialized search approach—effectively resulting in a machine learning problem being solved during compilation. The resultant compiler was developed by investing a 3 person-month effort, and resulted in up to 8× performance improvement on a production model, while also improving transparency during compilation.

## 6 Consequences of the MLIR Design

The MLIR design facilitates the modeling of new language and compilation abstractions while reusing existing, generic ones as well as their associated compilation methods. Effectively, the solution to many problems is to “add new ops, new types”, possibly collected into “a new dialect”. This is a significant design shift for compiler engineering. It produces new opportunities, challenges, and insights. This section explores a few of them.

### 6.1 Reusable compiler passes

The ability to represent multiple levels of abstraction in one IR creates the natural desire to write passes that work across multiple levels of abstraction. A common question about MLIR is “how do you write a compiler pass when you have openly extensible operations and type system?” While it is always possible for a compiler pass to treat unknown constructs in a conservatively correct way, our goal is to produce high performance code, so we need to do useful things in common cases. We have found four major approaches:

**Fundamental operation traits** Some “bread and butter” compiler passes like Dead Code Elimination and Common Subexpression Elimination rely on very simple properties (like “has no side effect” or “is commutative”) that we define as Op traits. The definition of an operation in ODS allows

在结构化中间表示中建模编程语言的高级语义能力非常强大。例如，对调度表的一流建模允许实现稳健的去虚拟化过程。虽然这可以通过定制编译器中间表示来实现，但使用MLIR使得Flang开发者能够将工程资源集中用于其领域的中间表示设计，而不是重新实现基础架构。

选择MLIR还解锁了其他并非Fortran特有的方言的可重用性：一种与语言无关的OpenMP方言可以在Fortran和C语言前端之间共享。同样，通过共享和重用面向GPU的方言和通道，在MLIR中使用OpenACC面向异构平台也变得可行。这之所以直接可行，得益于MLIR专门设计用于支持可组合方言的混合。

### 5.4 领域特定编译器

上述MLIR的应用场景都在大型编译工作流中，但它也被用于为特定小型工作流构建领域专用编译器。可复用且模块化的基础设施使得这些专用路径的构建变得可行且成本相对较低。

**优化MLIR模式重写** MLIR拥有一个可扩展的模式重写系统，如第4节所述。除了静态声明的模式外，我们的一些应用需要在运行时动态扩展重写模式，允许硬件供应商在驱动程序中添加新的低级优化。解决方案是将MLIR模式重写本身表达为一种MLIR方言，使我们能够利用MLIR基础设施动态构建和优化高效的有限状态机（FSM）匹配器和重写器。这项工作包含了其他系统中常见的FSM优化技术，例如LLVM SelectionDAG和GlobalISel指令选择系统。

**格点回归编译器** 格点回归[18]是一种以快速评估时间和可解释性著称的机器学习技术。该编译器的前身是使用C++模板实现的。这通过元编程实现了高性能代码，但在端到端模型上表达通用优化并不直接。该特定格点回归系统应用于拥有数百万用户的应用场景，因此性能提升至关重要。

MLIR被用作这一专业领域新编译器的基础，其驱动力来自一种专业化的搜索方法——这实际上使得编译过程中解决了一个机器学习问题。该编译器通过投入3人月的工作量得以开发，并在生产模型上实现了高达8×倍的性能提升，同时增强了编译过程的透明度。

## MLIR设计的六大影响

MLIR的设计便于对新的语言和编译抽象进行建模，同时复用现有的通用抽象及其相关的编译方法。实际上，许多问题的解决方案是“添加新的操作、新的类型”，可能收集到“一个新的方言”中。这是编译器工程的一个重大设计转变。它带来了新的机遇、挑战和见解。本节将探讨其中的几个方面。

### 6.1 可复用的编译器传递

在单一IR中表示多个抽象层次的能力，自然催生了编写跨多个抽象层次工作的编译过程的需求。关于MLIR的一个常见问题是：“当拥有开放可扩展的操作和类型系统时，如何编写编译器传递？”虽然编译器传递总可以以保守正确的方式处理未知结构，但我们的目标是生成高性能代码，因此需要在常见情况下进行有效处理。我们发现了四种主要方法：

**基础操作特性** 一些“基础且关键”的编译器传递（如死代码消除和公共子表达式消除）依赖于非常简单的属性（例如“无副作用”或“可交换性”），我们将这些属性定义为操作特性。ODS中操作的定义允许

the author of the operation to specify these traits, and passes can use this information to remain applicable across many different abstraction domains.

MLIR is extensible enough that it has a few structural properties, including information about whether an operation is known to be a control flow terminator, whether an operation containing a region is known to be isolated-from-above, etc. These allow generic modeling and processing of functions, closures, modules, and other code structures.

**Privileged operation hooks** While certain traits can be modeled with a single bit, others need C++ code to provide an implementation—constant folding logic for example. MLIR has first class support for certain hooks applicable to a large number of passes. These hooks can either be implemented on a per-operation basis, or in the Dialect object itself. The later approach is convenient for things like constant folding of TensorFlow ops, where delegation to existing logic is straightforward.

While constant folding is very important functionality, a more interesting hook is `getCanonicalizationPatterns`, which allows one to specify folding patterns that apply to an operation. This enables open extensibility of important algebraic simplifications (e.g.  $x - x \rightarrow 0$ ,  $\min(x, y, y) \rightarrow \min(x, y)$  etc.) and powers a common “Canonicalization” pass that can now be applied to all dialects. This allows this single extensible system to subsume things like “InstCombine”, “DAGCombine”, “PeepholeOptimizer”, “SILCombine”, and other special purpose passes seen in the LLVM ecosystem (and other compilers), which are a well-known maintenance and complexity burden.

**Optimization interfaces** A primary goal of MLIR is to allow open extensibility—not just in terms of operations and types, but also in terms of transformations. While canonicalization and constant folding are critical operations, there are a number of standard transformations that need to be parameterized in certain ways—e.g., to describe transformation specific properties, to implement cost models, etc.

The solution is a subsystem known as “Optimization Interfaces”. Consider the MLIR inlining pass: we would like the inliner to work on TensorFlow graphs, Flang functions, closures in a functional language etc.—but the inliner does not know what call sites or even the callees are! The core characteristics that an inliner needs to know are:

- whether it is valid to inline an operation into a given region;
- how to handle terminator operations that ended up in the middle of a block after inlining.

```
struct DialectInlinerInterface {  
    /// Returns true if the given operation 'op', that is registered to this  
    /// dialect, can be inlined into the given region, false otherwise.  
    bool isLegalToInline(Operation *, Region *, BlockAndValueMapping &) const;  
  
    /// Handle the given inlined terminator by replacing it with a new operation  
    /// as necessary.  
    void handleTerminator(Operation *, ArrayRef<Value *> ) const;  
};
```

Figure 10: Dialect interface to query legality of inlining.

In order to know these properties, the Inliner pass defines the interface in Figure 10. Individual operations and dialects may register their implementation of this interface with MLIR to benefit from the generic inliner pass. If an operation or dialect fails to provide an interface then the corresponding optimization pass will treat the operation conservatively. This design allows the implementer of a dialect to get up and running quickly, but derive more value out of the system by putting more implementation effort into interfaces like these over time.

Optimization interfaces also provide modularity benefit for the core compiler, because the dialect specific logic is implemented within the dialects themselves, instead of inside the core transformations.

**Dialect specific passes** Finally, it is valid and useful to define passes that are specific to particular dialects, which can be driven by full semantics of operations in the dialect(s) they are designed

操作的作者指定这些特性，而传递过程可以利用这些信息，以保持其在许多不同抽象领域中的适用性。

MLIR具有足够的可扩展性，因此它具备一些结构性属性，包括关于一个操作是否已知为控制流终结符、包含区域的操作是否已知为隔离于上层等信息。这些属性允许对函数、闭包、模块和其他代码结构进行通用建模和处理。

**特权操作钩子** 虽然某些特性可以用单个位来建模，但其他特性需要 C++ 代码来提供实现——例如常量折叠逻辑。MLIR 对适用于大量通道的特定钩子提供了一流的支持。这些钩子可以基于每个操作实现，也可以在方言对象本身中实现。后一种方法对于像 TensorFlow 操作的常量折叠这样的事情很方便，因为委托给现有逻辑是直接的。

尽管常量折叠是非常重要的功能，但更有趣的钩子是 `getCanonicalizationPatterns`，它允许指定应用于操作的折叠模式。这使得重要的代数简化（例如  $x - x \rightarrow 0$ ,  $\min(x, y, y) \rightarrow \min(x, y)$ ）等能够进行开放式扩展，并驱动一个通用的“规范化”过程，现在可以应用于所有方言。这使得这个单一的可扩展系统能够涵盖诸如“InstCombine”、“DAGCombine”、“PeepholeOptimizer”、“SILCombine”以及 LLVM 生态系统（和其他编译器）中出现的其他专用过程，这些都是众所周知的维护和复杂性负担。

**优化接口** MLIR 的主要目标之一是支持开放式扩展——不仅包括操作和类型，还包括转换。虽然规范化和常量折叠是关键操作，但许多标准转换需要以特定方式进行参数化——例如，用于描述转换的特定属性、实现成本模型等。

解决方案是一个被称为“优化接口”的子系统。以 MLIR 内联优化过程为例：我们希望内联器能够处理 TensorFlow 图、Flang 函数、函数式语言中的闭包等——但内联器并不知道调用点甚至被调用函数是什么！内联器需要了解的核心特性包括：

- 内联操作到给定区域是否有效；
- 如何处理内联后终止符操作落在块中间的情况。

```
struct DialectInlinerInterface {  
    /// 判断给定操作 'op'（注册到本方言）是否可合法内联至指定区域，返回 true  
    /// 表示可内联，false 表示不可内联。  
    bool isLegalToInline(Operation *, Region *, BlockAndValueMapping &) const;  
    /// 通过必要的新操作替换已内联的终止符来处理该终止符。  
    void handleTerminator(Operation *, ArrayRef<Value *> ) const;  
};
```

图10：查询内联合法性的方言接口。

为了了解这些属性，内联传递定义了图10中的接口。个别操作和方言可以向 MLIR 注册此接口的实现，以受益于通用内联传递。如果某个操作或方言未能提供接口，则相应的优化传递将保守地处理该操作。这种设计允许方言的实现者快速启动和运行，但随着时间的推移，通过在这些接口上投入更多实现努力，从系统中获得更多价值。

优化接口也为核心编译器带来了模块化的好处，因为方言特定的逻辑是在方言内部实现的，而不是在核心转换内部。

**方言特定传递** 最后，定义特定于特定方言的传递是有效且有用的，这些传递可以由它们所设计方言中操作的完整语义驱动。

for. These passes are just as useful in the MLIR system as they are in other compiler systems. For example, code generators that want to do custom scheduling of machine instructions based on particular machine constraints or other tricks that do not fit into a broader framework. This is a simple and useful starting point for new transformations, where generalization isn’t required.

### 6.2 Mixing dialects together

One of the most profound (but also most difficult to grok) aspects of MLIR is that it allows and encourages mixing operations from different dialects together into a single program. While certain cases of this are reasonably easy to understand (e.g. holding host and accelerator computation in the same module) the most interesting cases occur when dialects are directly mixed— because this enables an entire class of reuse that we have not seen in other systems.

Consider the affine dialect described in Section 5.2. The definition of affine control flow and affine mappings are independent of the semantics of the operations that are contained in affine regions. In our case, we combine the affine dialect with the “standard” dialect that represents simple arithmetic in a target independent form like LLVM IR, with multiple target-specific machine instruction dialects for internal accelerators. Others have combined it with abstractions from other problem domains.

The ability to reuse generic polyhedral transformations (using Op interfaces to get semantics of operations in specific transformations) is a powerful (and exciting to us) way of factoring compiler infrastructure. Another example is that an OpenMP dialect could be used and reused across a wide variety of source-language IRs.

### 6.3 Interoperability

Our work involves interoperation with a large number of existing systems, e.g., machine learning graphs encoded as protocol buffers, compiler IRs including LLVM IR, proprietary instruction sets, etc. Often the representation has a number of suboptimal or unfortunate decisions that made sense in the context of an existing system, but capabilities of MLIR enable a more expressive representation. Because importers and exporters are notoriously difficult to test (often the test cases are binary formats), we want to make sure their complexity is minimized.

The solution is to define a dialect that corresponds to the foreign system as directly as possible—allowing round tripping to-and-from that format in a simple and predictable way. Once the IR is imported into MLIR form, it can be raised and lowered to a more convenient IR using all of the MLIR infrastructure for doing these transformations, and allows those transformations to be tested similarly to all the other MLIR passes are.

There are numerous examples of such dialects, including: a) the LLVM dialect—which maps LLVM IR into MLIR, b) the representation of TensorFlow graphs—which is raised to ease analysis and transformations related to “switch and merge” nodes in TensorFlow, and c) functional-style control flow operators—“functional while” and “functional if” are common in machine learning graphs, in which it is more convenient to work with their bodies as regions instead of out-of-line functions.

This approach has worked well for us, and the MLIR tooling has also been useful to write tests for these foreign binary file formats.

### 6.4 Unopinionated design provides new challenges

While MLIR allows one to define almost arbitrary abstractions, it provides very little guidance on what *should* be done: what works better or worse in practice? We now have some experience with a number of engineers and researchers applying the techniques and technologies to new problem domains, and have realized that the “art” of compiler IR design and abstraction design is not well understood in the compiler and languages field—many people work within the constraints of established systems, but relatively few have had the opportunity define the abstractions themselves.

This is a challenge, but is also another set of opportunities for future research. The broader MLIR community is building a significant amount of expertise with these abstraction design trade-offs, and we expect this to be a fertile area of study over time.

这些过程在MLIR系统中与其他编译器系统中同样有用。例如，希望基于特定机器约束进行机器指令自定义调度的代码生成器，或者那些不适合更广泛框架的其他技巧。对于不需要泛化的新转换来说，这是一个简单而有用的起点。

### 6.2 混合方言

MLIR最深刻（但也最难理解）的一个方面是，它允许并鼓励将来自不同方言的操作混合到单个程序中。虽然某些情况相对容易理解（例如在同一模块中承载主机和加速器计算），但最有趣的情况发生在方言直接混合时——因为这实现了我们在其他系统中从未见过的整个重用类别。

考虑第5.2节中描述的仿射方言。仿射控制流和仿射映射的定义独立于仿射区域内包含的操作语义。在我们的案例中，我们将仿射方言与“标准”方言相结合，后者以目标独立形式（如LLVM IR）表示简单算术，并包含多个针对内部加速器的目标特定机器指令方言。其他研究者已将其与其他问题领域的抽象相结合。

重用通用多面体变换的能力（使用操作接口在特定变换中获取操作的语义）是一种强大的（且令我们兴奋的）编译器基础设施分解方式。另一个例子是，OpenMP方言可以在多种源语言中间表示中重复使用。

### 6.3 互操作性

我们的工作涉及与大量现有系统的互操作，例如编码为协议缓冲区的机器学习图、包括LLVM IR在内的编译器中间表示、专有指令集等。这些表示往往包含一些在现有系统背景下合理但并非最优或令人遗憾的设计决策，而MLIR的能力则允许更富表达力的表示。由于导入器和导出器的测试难度众所周知（测试用例通常是二进制格式），我们希望确保其复杂性降至最低。

解决方案是定义一个尽可能直接对应外部系统的方言——允许以简单且可预测的方式与该格式进行往返转换。一旦IR被导入到MLIR形式中，就可以利用MLIR的所有基础设施将其提升或降级为更便捷的IR，并使得这些转换能够像所有其他MLIR传递一样进行测试。

这类方言的例子有很多，包括：a) LLVM方言——将LLVM IR映射到MLIR中，b) TensorFlow图的表示——被提升以简化与TensorFlow中“switch和merge”节点相关的分析和转换，以及c) 函数式风格的控制流运算符——“函数式while”和“函数式if”在机器学习图中很常见，其中将它们作为区域而不是外部函数来处理更为方便。

这种方法对我们来说效果很好，MLIR工具在编写这些外部二进制文件格式的测试时也很有用。

### 6.4 无预设设计带来新挑战

虽然MLIR允许定义几乎任意的抽象，但它对*should*该做什么提供了很少的指导：在实践中哪些方法更好或更差？我们现在已经有了一些工程师和研究人员将这些技术应用于新问题领域的经验，并且意识到编译器IR设计和抽象设计的“艺术”在编译器和语言领域并未得到充分理解——许多人在现有系统的约束下工作，但相对较少的人有机会自己定义这些抽象。

这是一个挑战，但也是未来研究的另一系列机遇。更广泛的MLIR社区正在积累大量关于这些抽象设计权衡的专业知识，我们预计这将成为未来一个富有成果的研究领域。

## 6.5 Looking forward

The design of MLIR is different enough from other compiler infrastructures that we are still learning—even after building and applying it to many different systems. We believe that there is still a lot to discover, and several more years of research will be required until the design points are all fully understood and best practices are established. For example, the rise of out-of-tree dialects, increasing number of source language frontends using MLIR, possible application to Abstract Syntax Trees, and applications to structured data (like JSON, protocol buffers, etc) which are still very early and are likely to uncover interesting new challenges and opportunities.

## 7 Related Work

MLIR is a project that overlaps with multiple different domains. While the composed infrastructure provides a novel system, individual components have analogs in the literature. For references and discussion directly related to the IR design itself, please refer to Section 2.

MLIR is a compiler infrastructure akin to LLVM [25], but where LLVM has been a great boon to scalar optimizations and homogeneous compilation, MLIR aims to model a rich set of data structures and algorithms as first-class values and operations, including tensor algebra and algorithms, graph representations, as well as heterogeneous compilation. MLIR allows mix-and-match optimization decomposing compilation passes into components and redefining lowering, cleanup roles. This is largely attributed to the pattern rewriting infrastructure, capturing full-fledged transformations as a composition of small local patterns and controlling which pattern rewrites are applied at the granularity of an individual operation. Extending, formalizing, and verifying the rewriting logic automatically would be an important next step [9, 27]. On the backend side, MLIR’s DDR has an analogue to LLVM’s instruction selection infrastructure, supporting extensible operations with multi-result patterns and specification as constraints [49].

Numerous programming languages and models tackle hardware heterogeneity. Originally a homogeneous programming model, OpenMP added support for offloading tasks and parallel regions to accelerators [32], based on earlier proposals such as StarSs and OpenACC [34, 31]. C++ AMP, HCC and SyCL leverage a conventional Clang/LLVM flow and modern C++ to provide a high-level abstraction for hardware acceleration [46]. Unfortunately, all these examples very quickly lower high-level constructs to calls to a runtime execution environment, relying on pre-existing optimizations in the host language (typically C++) to alleviate the abstraction penalty. Far fewer efforts target the heterogeneous compilation process itself. Parallel intermediate representations extending LLVM IR address part of the issue but traditionally focus on the homogeneous setting [23, 42]. The most ambitious effort to date may be Liquid Metal [3], with a co-designed Domain Specific Language (DSL) and compilation flow converting managed object semantics into static, vector or reconfigurable hardware; yet most of the effort in its Lime compiler reside in fitting round objects into square hardware (paraphrasing Kou and Palsberg [24]). MLIR provides a direct embedding for high level languages embracing heterogeneity through extensible set of operations and types, while providing a common infrastructure for gradually lowering these constructs with maximal reuse of common components across the different targets.

Tackling language heterogeneity has been a long-term promise of metaprogramming systems, and of multistage programming in particular. Lightweight Modular Staging (LMS) [39] is a state of the art framework and runtime code generator, providing a library of core components for generating efficient code and embedding DSLs in Scala. Delite [45] promises dramatic productivity improvements for DSL developers, while supporting parallel and heterogeneous execution. We believe this approach is complementary to MLIR, providing a higher-level of abstraction to embed DSLs and implement optimizations through generic metaprogramming constructs.

One step further up into the language syntax, ANTLR [33] is among a class of parser generators that aim to make it easy to develop a new compiler frontend. MLIR currently does not have a general parser generation, no AST construction or modeling functionality. Combining MLIR with a system such as ANTLR could result in reusable compiler libraries from user input through to code generation.

More narrowly construed by their application to machine learning, XLA [57], Glow [40] and TVM [11], address similar heterogeneous compilation objectives. Yet these are rather specific code generation instances starting from a graph abstraction and targeting multi-dimensional vector

## 6.5 展望未来

MLIR的设计与其他编译器基础设施差异显著，即便在构建并将其应用于多种不同系统后，我们仍在持续探索。我们相信其中仍存在大量未知领域，需要再经过数年的研究才能完全理解其设计要点并确立最佳实践。例如，树外方言的兴起、使用MLIR的源语言前端数量不断增加、在抽象语法树上的潜在应用，以及对结构化数据（如JSON、协议缓冲区等）的应用探索仍处于早期阶段，这些领域很可能催生新的挑战与机遇。

## 7 相关工作

MLIR是一个与多个不同领域重叠的项目。虽然其组合基础设施提供了一个新颖的系统，但各个组件在文献中都有类似物。关于与IR设计本身直接相关的参考文献和讨论，请参阅第2节。

MLIR是一种类似于LLVM[25]的编译器基础设施，但LLVM主要推动了标量优化和同构编译的发展，而MLIR则致力于将丰富的数据结构和算法建模为一等值和操作，包括张量代数与算法、图表示以及异构编译。MLIR支持混合优化，将编译过程分解为组件并重新定义降级和清理角色。这主要归功于其模式重写基础设施，能够将完整的转换捕获为小型局部模式的组合，并在单个操作粒度上控制应用哪些模式重写。扩展、形式化并自动验证重写逻辑将是重要的下一步[9,27]。在后端方面，MLIR的DDR类似于LLVM的指令选择基础设施，支持具有多结果模式和约束规范的可扩展操作[49]。

众多编程语言和模型致力于应对硬件异构性。最初作为同构编程模型的OpenMP，基于StarSs和OpenACC等早期提案[32, 34, 31]，增加了将任务和并行区域卸载至加速器的支持。C++ AMP、HCC和SyCL利用传统的Clang/LLVM流程与现代C++，为硬件加速提供了高层抽象[46]。遗憾的是，这些方案都迅速将高层构造降级为运行时执行环境的调用，依赖宿主语言（通常是C++）的既有优化来缓解抽象开销。针对异构编译过程本身的研究则少得多。扩展LLVM IR的并行中间表示虽能解决部分问题，但传统上仍聚焦于同构场景[23, 42]。迄今为止最具雄心的尝试或许是Liquid Metal[3]，其通过协同设计的领域特定语言（DSL）与编译流程，将托管对象语义转换为静态、向量化或可重构硬件；然而其Lime编译器的主要精力仍耗费在“将圆形对象适配方形硬件”（转述Kou与Palsberg的表述[24]）。MLIR通过可扩展的操作与类型集合，为拥抱异构性的高级语言提供直接嵌入支持，同时建立通用基础设施，通过跨目标最大化复用公共组件，逐步实现这些构造的降级转换。

解决语言异构性一直是元编程系统，特别是多阶段编程的长期承诺。轻量级模块化分阶段（LMS）[39]是一个先进的框架和运行时代码生成器，提供了一套核心组件库，用于生成高效代码并在Scala中嵌入领域特定语言（DSL）。Delite [45]承诺为DSL开发者带来显著的生产力提升，同时支持并行和异构执行。我们认为这种方法与MLIR互补，提供了更高层次的抽象来嵌入DSL，并通过通用元编程结构实现优化。

在语言语法的更上一层，ANTLR [33]属于一类旨在简化新编译器前端开发的解析器生成器。MLIR目前不具备通用的解析器生成功能，也没有AST构建或建模功能。将MLIR与ANTLR等系统结合，可能实现从用户输入到代码生成的可复用编译器库。

更狭义地通过它们在机器学习中的应用来理解，XLA [57]、Glow [40] 和 TVM [11] 解决了类似的异构编译目标。然而，这些都是相当具体的代码生成实例，从图抽象出发，并针对多维向量



abstractions for accelerators. All of these could leverage MLIR as infrastructure, taking advantage of the common functionality while using their current code generation strategies. Similarly, the loop nest metaprogramming techniques from Halide [36] and TVM [11], earlier loop nest metaprogramming [19, 41, 5, 14], and fully automatic flows such as PolyMage [28], Tensor Comprehensions [52], Stripe [58], Diesel [16], Tiramisu [4] and their underlying polyhedral compilation techniques [17, 54, 8, 55] could co-exist as different code generation paths within an MLIR-based compilation framework. Serialization and interoperability formats, such as ONNX [48], have a different approach towards addressing the diversity of ML frontends by providing a common set of ops that different frameworks could map on to. ONNX would be a candidate as a dialect in MLIR to which other ops could be lowered to and from.

## 8 Conclusion and Future Work

We presented MLIR, a flexible and extensible infrastructure for compiler construction. This paper described MLIR’s concrete design, demonstrated its applicability to a range of important domains, and described a number of original research and engineering implications.

Looking ahead, we are eager to see how established compiler communities (e.g. the Clang C and C++ compiler) as well domain experts can benefit from the introduction of higher level, language-specific IRs. We are also interested to see if MLIR enables new approaches to teaching the art of compiler and IR design, and hope to see entirely new areas of research catalyzed or accelerated by this infrastructure.

**Future directions** There are multiple future directions being pursued for MLIR. In the ML and HPC space, these include inferring efficient Op implementations from reference rank-polymorphic specifications with symbolic shapes. It also involves enabling a wider range of data structures (sparse, graphs) and program transformations, bringing together symbolic reasoning, such as automatic differentiation and algorithmic simplification, with more conventional data flow and control flow-based optimizations. Beyond ML and HPC, one may consider MLIR’s applicability to other related domains, such as secure compilation, safety-critical systems, data analytics and graph processing, relational query optimization, etc.

Returning to the world of general-purpose languages, an obvious missing front-end is a C++ mid-level representation derived from Clang. Say, a “CIL” similar in spirit to Swift’s SIL and Rust’s MIR, that would facilitate the optimization of common C++ idioms that currently need to be reconstructed from lowered code (e.g., treating `std::vector` as an array rather than pointer manipulation). Supporting garbage-collected languages, higher-order and polymorphic type systems with type inference in MLIR are open challenges as well.

Exploring parallelism and concurrency constructs in LLVM has been difficult, primarily as the changes required are invasive and not easily layered (e.g., injecting metadata and inspecting all passes to ensure metadata is propagated while losing optimization opportunities as the level of abstraction is too low). With MLIR, parallel constructs can be first-class operations, using regions and parallel idiom-specific verification. This would support higher-level transformations before lowering to, e.g., LLVM where regular transformations can be performed on already lowered code.

Beyond debugging and testing, textual form of IR is also useful for education. Additional tooling to show the interaction of optimizations in high-performance compilation could demystify compilers for new students. IR design is an integral part of developing a new compiler or optimization framework, but many undergraduate compiler curricula do not cover IR design. MLIR provides opportunities for new approaches to such lessons that could be explored.

## References

- [1] M. Abadi, A. Agarwal, P. Barham, E. Brevdo, Z. Chen, C. Citro, G. S. Corrado, A. Davis, J. Dean, M. Devin, S. Ghemawat, I. Goodfellow, A. Harp, G. Irving, M. Isard, Y. Jia, R. Jozefowicz, L. Kaiser, M. Kudlur, J. Levenberg, D. Mané, R. Monga, S. Moore, D. Murray, C. Olah, M. Schuster, J. Shlens, B. Steiner, I. Sutskever, K. Talwar, P. Tucker, V. Vanhoucke, V. Vasudevan, F. Viégas, O. Vinyals, P. Warden, M. Wattenberg, M. Wicke, Y. Yu, and X. Zheng.

加速器的抽象。所有这些都可以利用MLIR作为基础设施，在利用其现有代码生成策略的同时，共享通用功能。类似地，来自Halide [36]和TVM [11]的循环嵌套元编程技术、早期的循环嵌套元编程研究[19, 41, 5, 14]，以及完全自动化的流程如PolyMage [28]、Tensor Comprehensions [52]、Stripe [58]、Diesel [16]、Tiramisu [4]及其底层的多面体编译技术[17, 54, 8, 55]，都可以作为不同的代码生成路径共存于基于MLIR的编译框架中。序列化和互操作性格式，如ONNX [48]，则采取了不同的方法来应对机器学习前端的多样性，它提供了一套通用的操作集，供不同框架映射使用。ONNX可以作为MLIR中的一个方言候选，其他操作可以降级到该方言或从该方言升级。

## 8 结论与未来工作

我们提出了MLIR，一个灵活且可扩展的编译器构建基础设施。本文阐述了MLIR的具体设计，展示了其在多个重要领域的适用性，并探讨了一系列原创性研究和工程意义。

展望未来，我们热切期待成熟的编译器社区（例如Clang C和C++编译器）以及领域专家如何从引入更高级别、语言特定的中间表示中受益。我们也关注MLIR是否能催生编译器与中间表示设计教学的新方法，并希望看到这一基础设施激发或加速全新的研究领域。

**未来方向** MLIR正朝着多个未来方向推进。在机器学习和高性能计算领域，这些方向包括：从具有符号形状的参考秩多态规范中推断高效算子实现；支持更广泛的数据结构（稀疏、图）和程序转换；将符号推理（如自动微分和算法简化）与更传统的数据流及基于控制流的优化相结合。除机器学习和高性能计算外，还可考虑MLIR在其他相关领域的适用性，例如安全编译、安全关键系统、数据分析与图处理、关系查询优化等。

回到通用语言的世界，一个明显缺失的前端是源自Clang的C++中级表示。例如，一种在精神上类似于Swift的SIL和Rust的MIR的“CIL”，它将促进对当前需要从低级代码重构的常见C++习语的优化（例如，将`std::vector`视为数组而非指针操作）。支持垃圾收集语言、高阶和多态类型系统以及MLIR中的类型推断，同样是开放的挑战。

探索LLVM中的并行性和并发性结构一直很困难，主要是因为所需的改动具有侵入性且不易分层（例如，注入元数据并检查所有传递以确保元数据得到传播，同时由于抽象层次过低而失去优化机会）。借助MLIR，并行结构可以作为一等操作，利用区域和并行惯用语的特定验证。这将支持在降级到例如LLVM之前进行更高级别的转换，在已降级的代码上执行常规转换。

除了调试和测试，中间表示（IR）的文本形式在教育中也很有用。额外的工具来展示高性能编译中优化的交互作用，可以为新学生揭开编译器的神秘面纱。IR设计是开发新编译器或优化框架不可或缺的一部分，但许多本科编译器课程并未涵盖IR设计。MLIR为这类课程提供了新方法的探索机会。

## 参考文献

- [1] M. Abadi, A. Agarwal, P. Barham, E. Brevdo, Z. Chen, C. Citro, G. S. Corrado, A. Davis, J. Dean, M. Devin, S. Ghemawat, I. Goodfellow, A. Harp, G. Irving, M. Isard, Y. Jia, R. Jozefowicz, L. Kaiser, M. Kudlur, J. Levenberg, D. Mané, R. Monga, S. Moore, D. Murray, C. Olah, M. Schuster, J. Shlens, B. Steiner, I. Sutskever, K. Talwar, P. Tucker, V. Vanhoucke, V. Vasudevan, F. Viégas, O. Vinyals, P. Warden, M. Wattenberg, M. Wicke, Y. Yu, and X. Zheng.

- TensorFlow: Large-scale machine learning on heterogeneous systems, 2015. Software available from tensorflow.org.
- [2] A. W. Appel. SSA is functional programming. *ACM SIGPLAN NOTICES*, 33(4):17–20, 1998.
- [3] J. Auerbach, D. F. Bacon, I. Burcea, P. Cheng, S. J. Fink, R. Rabbah, and S. Shukla. A compiler and runtime for heterogeneous computing. In *Proceedings of the 49th Annual Design Automation Conference*, DAC '12, pages 271–276, New York, NY, USA, 2012. ACM.
- [4] R. Baghdadi, J. Ray, M. B. Romdhane, E. Del Sozzo, A. Akkas, Y. Zhang, P. Suriana, S. Kamil, and S. Amarasinghe. Tiramisu: A polyhedral compiler for expressing fast and portable code. In *Proceedings of the 2019 IEEE/ACM International Symposium on Code Generation and Optimization*, CGO 2019, page 193–205. IEEE Press, 2019.
- [5] L. Bagnères, O. Zinenko, S. Huot, and C. Bastoul. Opening polyhedral compiler’s black box. In *Proceedings of the 2016 International Symposium on Code Generation and Optimization, CGO 2016, Barcelona, Spain, March 12-18, 2016*, pages 128–138, 2016.
- [6] G. Balakrishnan and T. Reps. Wysinwyx: What you see is not what you execute. *ACM Trans. Program. Lang. Syst.*, 32(6):23:1–23:84, Aug. 2010.
- [7] C. Bastoul. Code generation in the polyhedral model is easier than you think. In *Proceedings of the 13th International Conference on Parallel Architectures and Compilation Techniques*, PACT '04, pages 7–16, Washington, DC, USA, 2004. IEEE Computer Society.
- [8] U. Bondhugula, A. Hartono, J. Ramanujam, and P. Sadayappan. A practical automatic polyhedral parallelizer and locality optimizer. In *Proceedings of the ACM SIGPLAN 2008 Conference on Programming Language Design and Implementation, Tucson, AZ, USA, June 7-13, 2008*, pages 101–113, 2008.
- [9] M. Bravenboer, K. T. Kalleberg, R. Vermaas, and E. Visser. Stratego/xt 0.17. A language and toolset for program transformation. *Sci. Comput. Program.*, 72(1-2):52–70, 2008.
- [10] L. Chelini, O. Zinenko, T. Grosser, and H. Corporaal. Declarative loop tactics for domain-specific optimization. *TACO*, 16(4):55:1–55:25, 2020.
- [11] T. Chen, T. Moreau, Z. Jiang, L. Zheng, E. Yan, H. Shen, M. Cowan, L. Wang, Y. Hu, L. Ceze, C. Guestrin, and A. Krishnamurthy. TVM: An automated end-to-end optimizing compiler for deep learning. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*, pages 578–594, Carlsbad, CA, Oct. 2018. USENIX Association.
- [12] Y. Chen, A. Groce, C. Zhang, W. Wong, X. Z. Fern, E. Eide, and J. Regehr. Taming compiler fuzzers. In *ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI '13, Seattle, WA, USA, June 16-19, 2013*, pages 197–208, 2013.
- [13] C. Click and K. D. Cooper. Combining analyses, combining optimizations. *ACM Trans. Program. Lang. Syst.*, 17(2):181–196, Mar. 1995.
- [14] A. Cohen, S. Donadio, M.-J. Garzaran, C. Herrmann, O. Kiselyov, and D. Padua. In search of a program generator to implement generic transformations for high-performance computing. *Sci. Comput. Program.*, 62(1):25–46, Sept. 2006.
- [15] R. Cytron, J. Ferrante, B. K. Rosen, M. N. Wegman, and F. K. Zadeck. Efficiently computing static single assignment form and the control dependence graph. *ACM Trans. Program. Lang. Syst.*, 13(4):451–490, Oct. 1991.
- [16] V. Elango, N. Rubin, M. Ravishankar, H. Sandanagobalane, and V. Grover. Diesel: Dsl for linear algebra and neural net computations on gpus. In *Proceedings of the 2nd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, MAPL 2018, pages 42–51, New York, NY, USA, 2018. ACM.
- [17] P. Feautrier. Some efficient solutions to the affine scheduling problem. part II. multidimensional time. *Int. J. Parallel Program.*, 21(6):389–420, 1992.
- [18] E. Garcia and M. Gupta. Lattice regression. In Y. Bengio, D. Schuurmans, J. D. Lafferty, C. K. I. Williams, and A. Culotta, editors, *Advances in Neural Information Processing Systems 22*, pages 594–602. Curran Associates, Inc., 2009.
- [19] S. Girbal, N. Vasilache, C. Bastoul, A. Cohen, D. Parelo, M. Sigler, and O. Temam. Semi-automatic composition of loop transformations for deep parallelism and memory hierarchies. *Int. J. Parallel Program.*, 34(3):261–317, June 2006.

- TensorFlow: 异构系统上的大规模机器学习, 2015年。软件可从tensorflow.org获取。[2] A. W. Appel. SSA即函数式编程。 *ACM SIGPLAN NOTICES*, 33(4):17–20, 1998年。[3] J. Auerbach、D. F. Bacon、I. Burcea、P. Cheng、S. J. Fink、R. Rabbah和S. Shukla。异构计算的编译器和运行时。收录于*Proceedings of the 49th Annual Design Automation Conference*, DAC '12, 第271–276页, 美国纽约州纽约市, 2012年。ACM。[4] R. Baghdadi、J. Ray、M. B. Romdhane、E. Del Sozzo、A. Akkas、Y. Zhang、P. Suriana、S. Kamil和S. Amarasinghe。Tiramisu: 用于表达快速可移植代码的多面体编译器。收录于*Proceedings of the 2019 IEEE/ACM International Symposium on Code Generation and Optimization*, CGO 2019, 第193–205页。IEEE Press, 2019年。[5] L. Bagnères、O. Zinenko、S. Huot和C. Bastoul。揭开多面体编译器的黑盒。收录于*Proceedings of the 2016 International Symposium on Code Generation and Optimization, CGO 2016, Barcelona, Spain, March 12-18, 2016*, 第128–138页, 2016年。[6] G. Balakrishnan和T. Reps。所见非所执行: Wysinwyx. *ACM Trans. Program. Lang. Syst.*, 32(6):23:1–23:84, 2010年8月。[7] C. Bastoul。多面体模型中的代码生成比想象中简单。收录于*Proceedings of the 13th International Conference on Parallel Architectures and Compilation Techniques*, PACT '04, 第7–16页, 美国华盛顿特区, 2004年。IEEE计算机学会。[8] U. Bondhugula、A. Hartono、J. Ramanujam和P. Sadayappan。实用的自动多面体并行化与局部性优化器。收录于*Proceedings of the ACM SIGPLAN 2008 Conference on Programming Language Design and Implementation, Tucson, AZ, USA, June 7-13, 2008*, 第101–113页, 2008年。[9] M. Bravenboer、K. T. Kalleberg、R. Vermaas和E. Visser。Stratego/xt 0.17: 程序转换语言与工具集。 *Sci. Comput. Program.*, 72(1-2):52–70, 2008年。[10] L. Chelini、O. Zinenko、T. Grosser和H. Corporaal。面向领域特定优化的声明式循环策略。 *TACO*, 16(4):55:1–55:25, 2020年。[11] T. Chen、T. Moreau、Z. Jiang、L. Zheng、E. Yan、H. Shen、M. Cowan、L. Wang、Y. Hu、L. Ceze、C. Guestrin和A. Krishnamurthy。TVM: 面向深度学习的自动化端到端优化编译器。收录于*13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*, 第578–594页, 美国加利福尼亚州卡尔斯巴德, 2018年10月。USENIX协会。[12] Y. Chen、A. Groce、C. Zhang、W. Wong、X. Z. Fern、E. Eide和J. Regehr。驯服编译器模糊测试工具。收录于*ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI '13, Seattle, WA, USA, June 16-19, 2013*, 第197–208页, 2013年。[13] C. Click和K. D. Cooper。组合分析与组合优化。 *ACM Trans. Program. Lang. Syst.*, 17(2):181–196, 1995年3月。[14] A. Cohen、S. Donadio、M.-J. Garzaran、C. Herrmann、O. Kiselyov和D. Padua。寻找实现高性能计算通用变换的程序生成器。 *Sci. Comput. Program.*, 62(1):25–46, 2006年9月。[15] R. Cytron、J. Ferrante、B. K. Rosen、M. N. Wegman和F. K. Zadeck。高效计算静态单赋值形式与控制依赖图。 *ACM Trans. Program. Lang. Syst.*, 13(4):451–490, 1991年10月。[16] V. Elango、N. Rubin、M. Ravishankar、H. Sandanagobalane和V. Grover。Diesel: 面向GPU线性代数与神经网络计算的DSL。收录于*Proceedings of the 2nd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, MAPL 2018, 第42–51页, 美国纽约州纽约市, 2018年。ACM。[17] P. Feautrier。仿射调度问题的若干高效解法(第二部分): 多维时间。 *Int. J. Parallel Program.*, 21(6):389–420, 1992年。[18] E. Garcia和M. Gupta。格回归。收录于Y. Bengio、D. Schuurmans、J. D. Lafferty、C. K. I. Williams和A. Culotta编辑的*Advances in Neural Information Processing Systems 22*, 第594–602页。Curran Associates, Inc., 2009年。[19] S. Girbal、N. Vasilache、C. Bastoul、A. Cohen、D. Parelo、M. Sigler和O. Temam。面向深度并行与内存层次结构的半自动循环变换组合。 *Int. J. Parallel Program.*, 34(3):261–317, 2006年6月。

- [20] T. Grosser, A. Größlinger, and C. Lengauer. Polly - performing polyhedral optimizations on a low-level intermediate representation. *Parallel Processing Letters*, 22(4), 2012.
- [21] W. A. Havanki, S. Banerjia, and T. M. Conte. Treeegion scheduling for wide issue processors. In *Proceedings of the Fourth International Symposium on High-Performance Computer Architecture, Las Vegas, Nevada, USA, January 31 - February 4, 1998*, pages 266–276, 1998.
- [22] R. Johnson, D. Pearson, and K. Pingali. The program structure tree: Computing control regions in linear time. In *Proceedings of the ACM SIGPLAN 1994 Conference on Programming Language Design and Implementation*, PLDI '94, pages 171–185, New York, NY, USA, 1994. ACM.
- [23] D. Khaldi, P. Jouvelot, F. Irigoin, C. Ancourt, and B. Chapman. LLVM parallel intermediate representation: Design and evaluation using OpenSHMEM communications. In *Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC*, LLVM '15, pages 2:1–2:8, New York, NY, USA, 2015. ACM.
- [24] S. Kou and J. Palsberg. From oo to fpga: Fitting round objects into square hardware? In *Proceedings of the ACM International Conference on Object Oriented Programming Systems Languages and Applications*, OOPSLA '10, pages 109–124, New York, NY, USA, 2010. ACM.
- [25] C. Lattner and V. Adve. LLVM: A compilation framework for lifelong program analysis & transformation. In *Proceedings of the International Symposium on Code Generation and Optimization: Feedback-directed and Runtime Optimization*, CGO '04, pages 75–, Washington, DC, USA, 2004. IEEE Computer Society.
- [26] T. Lindholm and F. Yellin. *Java Virtual Machine Specification*. Addison-Wesley Longman Publishing Co., Inc., Boston, MA, USA, 2nd edition, 1999.
- [27] J. Meseguer. Twenty years of rewriting logic. In *Proceedings of the 8th International Conference on Rewriting Logic and Its Applications*, WRLA'10, pages 15–17, Berlin, Heidelberg, 2010. Springer-Verlag.
- [28] R. T. Mullapudi, V. Vasista, and U. Bondhugula. PolyMage: Automatic optimization for image processing pipelines. In *International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pages 429–443, 2015.
- [29] G. C. Necula. Translation validation for an optimizing compiler. *SIGPLAN Not.*, 35(5):83–94, May 2000.
- [30] Open64 Developers. Open64 compiler and tools, 2001.
- [31] OpenACC application programming interface. Online, <https://www.openacc.org>, accessed Feb 19, 2020.
- [32] OpenMP ARB. The OpenMP API specification for parallel programming. Online, <https://www.openmp.org>, accessed Feb 19, 2020.
- [33] T. J. Parr and R. W. Quong. Antlr: A predicated-ll(k) parser generator. *Softw. Pract. Exper.*, 25(7):789–810, July 1995.
- [34] J. Planas, R. M. Badia, E. Ayguadé, and J. Labarta. Hierarchical task-based programming with starss. *IJHPCA*, 23(3):284–299, 2009.
- [35] A. Pnueli, M. Siegel, and E. Singerman. Translation validation. In *Tools and Algorithms for Construction and Analysis of Systems, 4th International Conference, TACAS '98, Held as Part of the European Joint Conferences on the Theory and Practice of Software, ETAPS'98, Lisbon, Portugal, March 28 - April 4, 1998, Proceedings*, pages 151–166, 1998.
- [36] J. Ragan-Kelley, A. Adams, D. Sharlet, C. Barnes, S. Paris, M. Levoy, S. Amarasinghe, and F. Durand. Halide: Decoupling algorithms from schedules for high-performance image processing. *Commun. ACM*, 61(1):106–115, Dec. 2017.
- [37] G. Ramalingam. On loops, dominators, and dominance frontiers. *ACM Trans. Program. Lang. Syst.*, 24(5):455–490, 2002.
- [38] C. Reddy and U. Bondhugula. Effective automatic computation placement and data allocation for parallelization of regular programs. In *Proceedings of the 28th ACM International Conference on Supercomputing*, ICS '14, pages 13–22, New York, NY, USA, 2014. ACM.
- [39] T. Rompf and M. Odersky. Lightweight modular staging: a pragmatic approach to runtime code generation and compiled dsls. *Commun. ACM*, 55(6):121–130, 2012.

- [20] T. Grosser、A. Größlinger 和 C. Lengauer。Polly - 在低级中间表示上执行多面体优化。*Parallel Processing Letters*, 22(4), 2012年。[21] W. A. Havanki、S. Banerjia 和 T. M. Conte。面向宽发射处理器的树区域调度。收录于 *Proceedings of the Fourth International Symposium on High-Performance Computer Architecture, Las Vegas, Nevada, USA, January 31 - February 4, 1998*, 第266–276页, 1998年。[22] R. Johnson、D. Pearson 和 K. Pingali。程序结构树：在线性时间内计算控制区域。收录于 *Proceedings of the ACM SIGPLAN 1994 Conference on Programming Language Design and Implementation*, PLDI '94, 第171–185页, 美国纽约, 1994年。ACM。[23] D. Khaldi、P. Jouvelot、F. Irigoin、C. Ancourt 和 B. Chapman。LLVM并行中间表示：使用OpenSHMEM通信的设计与评估。收录于 *Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC*, LLVM '15, 第2:1–2:8页, 美国纽约, 2015年。ACM。[24] S. Kou 和 J. Palsberg。从OO到FPGA：将圆形对象放入方形硬件？收录于 *Proceedings of the ACM International Conference on Object Oriented Programming Systems Languages and Applications*, OOPSLA '10, 第109–124页, 美国纽约, 2010年。ACM。[25] C. Lattner 和 V. Adve。LLVM：一个用于终身程序分析与转换的编译框架。收录于 *Proceedings of the International Symposium on Code Generation and Optimization: Feedback-directed and Runtime Optimization*, CGO '04, 第75页起, 美国华盛顿特区, 2004年。IEEE计算机学会。[26] T. Lindholm 和 F. Yellin。*Java Virtual Machine Specification*。Addison-Wesley Longman出版公司, 美国马萨诸塞州波士顿, 第二版, 1999年。[27] J. Meseguer。重写逻辑二十年。收录于 *Proceedings of the 8th International Conference on Rewriting Logic and Its Applications*, WRLA'10, 第15–17页, 德国柏林海德堡, 2010年。Springer-Verlag。[28] R. T. Mullapudi、V. Vasista 和 U. Bondhugula。PolyMage：图像处理管道的自动优化。收录于 *International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 第429–443页, 2015年。[29] G. C. Necula。优化编译器的翻译验证。*SIGPLAN Not.*, 35(5):83–94, 2000年5月。[30] Open64开发者。Open64编译器与工具, 2001年。[31] OpenACC应用程序编程接口。在线, <https://www.openacc.org>, 访问于2020年2月19日。[32] OpenMP ARB。用于并行编程的OpenMP API规范。在线, <https://www.openmp.org>, 访问于2020年2月19日。[33] T. J. Parr 和 R. W. Quong。Antlr：一个谓词LL(k)解析器生成器。*Softw. Pract. Exper.*, 25(7):789–810, 1995年7月。[34] J. Planas、R. M. Badia、E. Ayguadé 和 J. Labarta。基于星形系统的层次化任务编程。*IJHPCA*, 23(3):284–299, 2009年。[35] A. Pnueli、M. Siegel 和 E. Singerman。翻译验证。收录于 *Tools and Algorithms for Construction and Analysis of Systems, 4th International Conference, TACAS '98, Held as Part of the European Joint Conferences on the Theory and Practice of Software, ETAPS'98, Lisbon, Portugal, March 28 - April 4, 1998, Proceedings*, 第151–166页, 1998年。[36] J. Ragan-Kelley、A. Adams、D. Sharlet、C. Barnes、S. Paris、M. Levoy、S. Amarasinghe 和 F. Durand。Halide：将算法与调度解耦以实现高性能图像处理。*Commun. ACM*, 61(1):106–115, 2017年12月。[37] G. Ramalingam。论循环、支配者与支配前沿。*ACM Trans. Program. Lang. Syst.*, 24(5):455–490, 2002年。[38] C. Reddy 和 U. Bondhugula。面向规则程序并行化的有效自动计算放置与数据分配。收录于 *Proceedings of the 28th ACM International Conference on Supercomputing*, ICS '14, 第13–22页, 美国纽约, 2014年。ACM。[39] T. Rompf 和 M. Odersky。轻量级模块化阶段化：一种实用的运行时代码生成与编译DSL方法。*Commun. ACM*, 55(6):121–130, 2012年。

- [40] N. Rotem, J. Fix, S. Abdulrasool, G. Catron, S. Deng, R. Dzhabarov, N. Gibson, J. Hegeman, M. Lele, R. Levenstein, J. Montgomery, B. Maher, S. Nadathur, J. Olesen, J. Park, A. Rakhov, M. Smelyanskiy, and M. Wang. Glow: Graph lowering compiler techniques for neural networks, 2018.
- [41] G. Rudy, M. M. Khan, M. Hall, C. Chen, and J. Chame. A programming language interface to describe transformations and code generation. In K. Cooper, J. Mellor-Crummey, and V. Sarkar, editors, *Languages and Compilers for Parallel Computing*, pages 136–150, Berlin, Heidelberg, 2011. Springer Berlin Heidelberg.
- [42] T. B. Schardl, W. S. Moses, and C. E. Leiserson. Tapir: Embedding fork-join parallelism into LLVM’s intermediate representation. *SIGPLAN Not.*, 52(8):249–265, Jan. 2017.
- [43] B. Schommer, C. Cullmann, G. Gebhard, X. Leroy, M. Schmidt, and S. Wegener. Embedded Program Annotations for WCET Analysis. In *WCET 2018: 18th International Workshop on Worst-Case Execution Time Analysis*, volume 63, Barcelona, Spain, July 2018. Dagstuhl Publishing.
- [44] E. Schweitz. An MLIR dialect for high-level optimization of fortran. LLVM Developer Meeting, Oct 2019.
- [45] A. K. Sujeeth, K. J. Brown, H. Lee, T. Rompf, H. Chafi, M. Odersky, and K. Olukotun. Delite: A compiler architecture for performance-oriented embedded domain-specific languages. *ACM Trans. Embedded Comput. Syst.*, 13(4s):134:1–134:25, 2014.
- [46] SyCL: C++ single-source heterogeneous programming for OpenCL. Online, <https://www.khronos.org/sycl>, accessed Feb 19, 2020.
- [47] TableGen - LLVM 10 Documentation. Online, <https://llvm.org/docs/TableGen/>, accessed Nov 22, 2019, 2019.
- [48] The Linux Foundation. ONNX: Open neural network exchange. Online, <https://github.com/onnx/onnx>, accessed Feb 19, 2020.
- [49] P. Thier, M. A. Ertl, and A. Krall. Fast and flexible instruction selection with constraints. In *Proceedings of the 27th International Conference on Compiler Construction*, CC 2018, pages 93–103, New York, NY, USA, 2018. ACM.
- [50] J. Tristan and X. Leroy. Formal verification of translation validators: a case study on instruction scheduling optimizations. In *Proceedings of the 35th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2008, San Francisco, California, USA, January 7-12, 2008*, pages 17–27, 2008.
- [51] J. Tristan and X. Leroy. Verified validation of lazy code motion. In *Proceedings of the 2009 ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI 2009, Dublin, Ireland, June 15-21, 2009*, pages 316–326, 2009.
- [52] N. Vasilache, O. Zinenko, T. Theodoridis, P. Goyal, Z. Devito, W. S. Moses, S. Verdoolaege, A. Adams, and A. Cohen. The next 700 accelerated layers: From mathematical expressions of network computation graphs to accelerated GPU kernels, automatically. *ACM Trans. Archit. Code Optim.*, 16(4):38:1–38:26, Oct. 2019.
- [53] A. Veen. Dataflow machine architecture. *ACM Comput. Surv.*, 18:365–396, 12 1986.
- [54] S. Verdoolaege. ISL: An integer set library for the polyhedral model. In *Proceedings of the Third International Congress Conference on Mathematical Software*, ICMS’10, pages 299–302, Berlin, Heidelberg, 2010. Springer-Verlag.
- [55] S. Verdoolaege, J. Carlos Juega, A. Cohen, J. Ignacio Gómez, C. Tenllado, and F. Catthoor. Polyhedral parallel code generation for CUDA. *ACM Trans. Archit. Code Optim.*, 9(4):54:1–54:23, Jan. 2013.
- [56] S. T. Vu, K. Heydemann, A. de Grandmaison, and A. Cohen. Secure delivery of program properties through optimizing compilation. In *ACM SIGPLAN 2020 International Conference on Compiler Construction (CC 2020)*, San Diego, CA, Feb. 2020.
- [57] XLA - TensorFlow, compiled. Google Developers Blog, <https://developers.googleblog.com/2017/03/xla-tensorflow-compiled.html>, Mar 2017.
- [58] T. Zerrell and J. Bruestle. Stripe: Tensor compilation via the nested polyhedral model. *CoRR*, abs/1903.06498, 2019.

- [40] N. Rotem, J. Fix, S. Abdulrasool, G. Catron, S. Deng, R. Dzhabarov, N. Gibson, J. Hegeman, M. Lele, R. Levenstein, J. Montgomery, B. Maher, S. Nadathur, J. Olesen, J. Park, A. Rakhov, M. Smelyanskiy 与 M. Wang. Glow: 面向神经网络的图降级编译技术, 2018 年。[41] G. Rudy, M. M. Khan, M. Hall, C. Chen 与 J. Chame。一种用于描述变换与代码生成的编程语言接口。载于 K. Cooper, J. Mellor-Crummey 与 V. Sarkar 编, *Languages and Compilers for Parallel Computing*, 第136–150页, 柏林, 海德堡, 2011年。Springer Berlin Heidelberg。[42] T. B. Schardl, W. S. Moses 与 C. E. Leiserson。Tapir: 将 fork-join 并行性嵌入 LLVM 中间表示。 *SIGPLAN Not.*, 52(8):249–265, 2017年1月。[43] B. Schommer, C. Cullmann, G. Gebhard, X. Leroy, M. Schmidt 与 S. Wegener。面向最坏情况执行时间分析的嵌入式程序标注。载于 *WCET 2018: 18th International Workshop on Worst-Case Execution Time Analysis*, 第63卷, 西班牙巴塞罗那, 2018年7月。Dagstuhl Publishing。[44] E. Schweitz。一种用于 Fortran 高级优化的 MLIR 方言。LLVM 开发者大会, 2019年10月。[45] A. K. Sujeeth, K. J. Brown, H. Lee, T. Rompf, H. Chafi, M. Odersky 与 K. Olukotun。Delite: 一种面向性能的嵌入式领域特定语言的编译器架构。 *ACM Trans. Embedded Comput. Syst.*, 13(4s):134:1–134:25, 2014年。[46] SyCL: 面向 OpenCL 的 C++ 单源异构编程。在线, <https://www.khronos.org/sycl>, 访问于2020年2月19日。[47] TableGen - LLVM 10 文档。在线, <https://llvm.org/docs/TableGen/>, 访问于2019年11月22日, 2019 年。[48] Linux 基金会。ONNX: 开放神经网络交换格式。在线, <https://github.com/onnx/onnx>, 访问于2020年2月19日。[49] P. Thier, M. A. Ertl 与 A. Krall。基于约束的快速灵活指令选择。载于 *Proceedings of the 27th International Conference on Compiler Construction*, CC 2018, 第93–103页, 美国纽约, 2018年。ACM。[50] J. Tristan 与 X. Leroy。翻译验证器的形式化验证: 以指令调度优化为例。载于 *Proceedings of the 35th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2008, San Francisco, California, USA, January 7-12, 2008*, 第17–27页, 2008年。[51] J. Tristan 与 X. Leroy。惰性代码移动的已验证验证。载于 *Proceedings of the 2009 ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI 2009, Dublin, Ireland, June 15-21, 2009*, 第316–326页, 2009年。[52] N. Vasilache, O. Zinenko, T. Theodoridis, P. Goyal, Z. Devito, W. S. Moses, S. Verdoolaege, A. Adams 与 A. Cohen。接下来的700个加速层: 从网络计算图的数学表达式到自动生成的加速 GPU 内核。 *ACM Trans. Archit. Code Optim.*, 16(4):38:1–38:26, 2019年10月。[53] A. Veen。数据流机器架构。 *ACM Comput. Surv.*, 18:365–396, 1986年12月。[54] S. Verdoolaege。ISL: 多面体模型的整数集库。载于 *Proceedings of the Third International Congress Conference on Mathematical Software*, ICMS’10, 第299–302页, 柏林, 海德堡, 2010年。Springer-Verlag。[55] S. Verdoolaege, J. Carlos Juega, A. Cohen, J. Ignacio Gómez, C. Tenllado 与 F. Catthoor。面向 CUDA 的多面体并行代码生成。 *ACM Trans. Archit. Code Optim.*, 9(4):54:1–54:23, 2013年1月。[56] S. T. Vu, K. Heydemann, A. de Grandmaison 与 A. Cohen。通过优化编译安全传递程序属性。载于 *ACM SIGPLAN 2020 International Conference on Compiler Construction (CC 2020)*, 美国加州圣地亚哥, 2020年2月。[57] XLA - TensorFlow, 已编译。Google 开发者博客, <https://developers.googleblog.com/2017/03/xla-tensorflow-compiled.html>, 2017年3月。[58] T. Zerrell 与 J. Bruestle。Stripe: 基于嵌套多面体模型的张量编译。 *CoRR*, abs/1903.06498, 2019年。